Day 1 — Setup Guide

Project Setup & Docker

event-management-system | Sprint 1 | Windows | Beginner Guide

Student	Abdul Arfath
Reg No	P05MG24S038006
Goal today	Docker container running Flask on localhost:5000
Time estimate	3–4 hours (including downloads)
Phases	6 phases	Install → Git → Structure → Code → Docker → Commit
End result	Browser shows: Hello from EMS! Docker is working.

Overview — What we are doing today

Today is purely about setting up your development environment and creating the skeleton of your project. You will not write any feature code today — that starts on Day 2.

By the end of today you will have:

- Git installed and configured with your GitHub account
- Python 3.11 installed on Windows
- Docker Desktop installed and running
- A professional project folder structure inside VS Code
- 5 project files written: Dockerfile, docker-compose.yml, requirements.txt, .env.example, app.py
- A Docker container that builds and runs your Flask app at http://localhost:5000
- Everything committed and pushed to GitHub

> **Why Docker?**
> On Day 3 you will add Firebase. On Day 10 you will add PayPal. Docker keeps all of this packaged together so it runs the same way on your machine, your internship office machine, and eventually any server — without installing anything extra.

The 6 phases today

Phase	Name	What you will do	Time
1	**Install tools**	Install Git, Python 3.11, and Docker Desktop on Windows	~60 min
2	**Configure Git & GitHub**	Set your name/email in Git, clone your repo to your PC	~20 min
3	**Create folder structure**	Build the project skeleton inside VS Code	~15 min
4	**Write project files**	Write 5 files: Dockerfile, docker-compose.yml, requirements.txt, .env.example, app.py	~40 min
5	**Build & run Docker**	Build the image, start the container, open browser to verify	~20 min
6	**Commit & push to GitHub**	Stage all files, write a commit message, push to GitHub	~15 min

> **Before you start**
> Open VS Code now. You will use it as your text editor throughout today. Every file you create will be opened and edited inside VS Code.

Phase 1 of 6

Install Git, Python & Docker Desktop

Step 1/6 Install Git (includes Git Bash — your terminal for the whole project)

What is Git?

Git is a tool that saves snapshots of your code over time. Git Bash is a terminal (command line) that comes with it — it lets you type commands to control your project. We use Git Bash instead of Command Prompt because it uses Linux-style commands, which match every tutorial and guide you will ever find online.

How to install:

- Open your browser and go to: https://git-scm.com/download/win
- Click the first download link — it says "Click here to download" and auto-detects your Windows version
- Once downloaded, double-click the installer file (it will be named something like Git-2.x.x-64-bit.exe)

> **Installer settings — important**
> On every screen of the installer, keep the default options selected and just click Next. The only screen you need to change is "Choosing the default editor used by Git" — change it from Vim to "Use Visual Studio Code as Git's default editor".

- Continue clicking Next through all remaining screens, then click Install
- When installation finishes, click Finish

Verify Git installed correctly:

Press the Windows key on your keyboard, type Git Bash, and press Enter. A black terminal window will open. Type the following command exactly and press Enter:

```
git --version
```

You should see something like:

```
git version 2.45.2.windows.1
```

> **If you see an error**
> Close Git Bash, restart Windows, open Git Bash again, and try the command again. If it still fails, re-run the Git installer.

Step 2/6 Install Python 3.11

What is Python?

Python is the programming language your EMS is written in. We install it on Windows so that when Docker is not running, you can still run Python scripts directly. It is also needed by some tools we use.

How to install:

- Open your browser and go to: https://www.python.org/downloads/release/python-3119/
- Scroll to the bottom of the page to the "Files" section
- Click: Windows installer (64-bit) — this downloads a file called python-3.11.9-amd64.exe
- Once downloaded, double-click the installer

> **CRITICAL — tick this box first**
> On the very first screen of the Python installer, before clicking anything else, you MUST tick the checkbox at the bottom that says "Add Python 3.11 to PATH". If you miss this, Python commands will not work in Git Bash.

- After ticking Add Python to PATH, click Install Now
- When it finishes, click Close

Verify Python installed correctly:

In Git Bash, type:

```
python --version
```

You should see:

```
Python 3.11.9
```

Step 3/6 Install Docker Desktop

What is Docker?

Docker packages your entire application — Python, Flask, all libraries, all config — into a single box called a container. You press one command and the whole app starts, the same way, every time. Think of it like a lunchbox: everything your app needs is packed inside it.

Before installing — enable WSL 2:

Docker on Windows requires WSL 2 (Windows Subsystem for Linux). This is a feature built into Windows 10/11. We need to enable it first.

Open Git Bash and run these two commands one at a time (press Enter after each):

```
wsl --install

wsl --set-default-version 2
```

> **Restart required**
> After running these commands Windows will ask you to restart. Save any open work and restart your PC. After restarting, open Git Bash again — a Ubuntu terminal may also open automatically, that is normal, you can close it.

Now install Docker Desktop:

- Open your browser and go to: https://www.docker.com/products/docker-desktop/
- Click the blue Download Docker Desktop button — it auto-detects Windows
- Once downloaded, double-click Docker Desktop Installer.exe
- On the installer screen, make sure Use WSL 2 instead of Hyper-V is ticked
- Click OK and wait — installation takes 3–5 minutes
- When it finishes, click Close and restart (it will prompt you)

Start Docker Desktop:

- After restarting, find Docker Desktop in your Start menu and open it
- Accept the service agreement (click Accept)
- Docker will show a spinning animation while it starts — wait for it to say Engine running at the bottom left

> **Docker must be running**
> Docker Desktop must be open and showing Engine running every time you work on this project. If Docker is not running, the docker commands will fail.

Verify Docker installed correctly:

In Git Bash, type:

```
docker --version
```

You should see something like:

```
Docker version 26.1.3, build b72abbb
```

Also run:

```
docker run hello-world
```

Docker will download a tiny test image and print a message that starts with Hello from Docker! — this confirms Docker is working.

Phase 2 of 6

Configure Git & Create GitHub Repository

Step 4/6 Configure your Git identity and create the GitHub repo

Set your name and email in Git:

Git records your name and email on every commit you make. Run these two commands in Git Bash — replace the values with your actual name and the email you used for GitHub:

```
git config --global user.name "Abdul Arfath"
git config --global user.email "your@email.com"
```

Verify it was saved:

```
git config --list
```

You should see your name and email printed in the output.

Create the GitHub repository:

- Open your browser and go to: https://github.com
- Log in to your account
- Click the green New button (top left, next to your repositories list)
- Fill in the form:
  - Repository name: event-management-system
  - Description: B2B Event Management System — Flask, Firebase, Docker
  - Visibility: Private (you can make it public later)
  - Tick: Add a README file
  - Add .gitignore: choose Python from the dropdown
- Click Create repository

Clone the repository to your PC:

Now we bring the repository from GitHub down to your Windows PC. In Git Bash, run:

```
cd ~/Documents
git clone https://github.com/YOUR-USERNAME/event-management-system.git
cd event-management-system
```

Replace YOUR-USERNAME

> In the clone command above, replace YOUR-USERNAME with your actual GitHub username. You can find it on your GitHub profile page.

Create and switch to a develop branch:

We keep the main branch clean and do all development on a branch called develop. Run:

```
git checkout -b develop
```

You should see: Switched to a new branch 'develop'

Open the project in VS Code:

```
code .
```

VS Code will open with your project folder. You will see the README.md and .gitignore files GitHub created. All file editing from now on happens inside VS Code.

Phase 3 of 6

Create the Project Folder Structure

Step 5/6 Create folders and placeholder files

The folder structure we are building:

```
event-management-system/
├── app/
│   └── __init__.py
├── templates/
│   └── .gitkeep
├── static/
│   └── .gitkeep
├── tests/
│   └── .gitkeep
├── app.py
├── requirements.txt
├── Dockerfile
├── docker-compose.yml
├── .env.example
├── .gitignore
└── README.md
```

What each folder is for:

Folder / File	Purpose
app/	Python package — all your Flask routes and modules will go here as the project grows
app/__init__.py	Makes app/ a Python package — empty for now, will hold the Flask app factory later
templates/	HTML files (the web pages users see) — Flask looks here automatically
static/	CSS, JavaScript, and image files — served directly to the browser
tests/	Unit tests — you'll add tests here from Day 3 onwards
app.py	The entry point — runs the Flask app. This is what Docker starts
requirements.txt	The list of Python libraries your app needs — Docker reads this to install them
Dockerfile	Instructions for Docker on how to build your app into a container image
docker-compose.yml	Defines and runs your container with one command: docker compose up
.env.example	A template showing which secret keys are needed — safe to commit to Git
.env	The actual secret keys (created from .env.example) — NEVER committed to Git

Run these commands in Git Bash to create the folders:

Make sure you are inside the event-management-system folder (Git Bash shows the folder name in the prompt). Then run:

```
mkdir app templates static tests
touch app/__init__.py
touch templates/.gitkeep static/.gitkeep tests/.gitkeep
touch app.py requirements.txt Dockerfile docker-compose.yml .env.example
```

> **What is .gitkeep?**
> Git does not track empty folders. The .gitkeep file is a blank placeholder file that forces Git to include the empty folder in your repository. You will delete it when you add real files to those folders.

Now check your VS Code Explorer panel (left sidebar). You should see all the folders and files created. If anything is missing, run the touch command again for the missing file.

Phase 4 of 6

Write the 5 Project Files

Step **6/6** **Write each file exactly as shown below**

Open each file in VS Code by clicking it in the Explorer panel. Copy the content shown below exactly into each file, then press Ctrl+S to save.

File 1 of 5 — requirements.txt

This file lists every Python library your project needs. Docker reads this file and installs all of them automatically when building the container.

```
# Web framework
Flask==3.0.3

# Firebase Admin SDK — connects to Firestore and Firebase Auth
firebase-admin==6.5.0

# PayPal SDK — payment processing
paypalrestsdk==1.13.3

# SendGrid — sending confirmation emails
sendgrid==6.11.0

# QR code generation
qrcode[pil]==7.4.2

# Environment variable loading from .env file
python-dotenv==1.0.1

# CSRF protection for forms
Flask-WTF==1.2.1
```

> **Why list all libraries now?**
> We list every library the finished project needs from day one. This means Docker always builds the complete environment. You will not need to add to this file until you add a library not in this list.

File 2 of 5 — .env.example

This file is a template that shows every secret key the project needs. It stores fake placeholder values — never real keys. It is safe to commit to GitHub.

When you set up Firebase, PayPal, and SendGrid in later days, you will create a real .env file by copying this and filling in the actual values.

```
# ─── Flask ──────────────────────────────────────────────────────────
FLASK_ENV=development
FLASK_DEBUG=1
SECRET_KEY=your-flask-secret-key-change-this

# ─── Firebase ───────────────────────────────────────────────────────
FIREBASE_CREDENTIALS_PATH=firebase-key.json
FIREBASE_PROJECT_ID=your-firebase-project-id
FIREBASE_STORAGE_BUCKET=your-project-id.appspot.com

# ─── PayPal ─────────────────────────────────────────────────────────
PAYPAL_MODE=sandbox
PAYPAL_CLIENT_ID=your-paypal-sandbox-client-id
PAYPAL_CLIENT_SECRET=your-paypal-sandbox-client-secret

# ─── SendGrid ───────────────────────────────────────────────────────
SENDGRID_API_KEY=your-sendgrid-api-key
SENDGRID_FROM_EMAIL=noreply@yourdomain.com

# ─── QR Code ────────────────────────────────────────────────────────
QR_HMAC_SECRET=your-qr-hmac-secret-key-change-this
```

File 3 of 5 — app/__init__.py

This file is empty for now. It simply tells Python that the app folder is a package (a collection of code files that belong together). We will fill it in on Day 3 when we build the Flask app factory.

```
# Flask application package
# This file will contain the create_app() factory function from Day 3 onwards
```

File 4 of 5 — app.py

This is the entry point of your application — the file Docker runs to start your web server. Right now it creates a minimal Flask app with a single route that returns a test message. We will expand this heavily starting Day 3.

```
import os
from flask import Flask, jsonify
from dotenv import load_dotenv

# Load environment variables from .env file
load_dotenv()

# Create the Flask application instance
app = Flask(__name__)
app.config['SECRET_KEY'] = os.getenv('SECRET_KEY', 'dev-secret')

# ── Health check route ──────────────────────────────────────────────
@app.route('/')
def index():
    return jsonify({
```

```
        'message': 'Hello from EMS! Docker is working.',
        'status':  'ok',
        'project': 'event-management-system',
        'day':     1,
        'sprint':  'Sprint 1 — Foundation'
    })

# ── Entry point ──────────────────────────────────────────────────────
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```

> **host='0.0.0.0' explained**
> This tells Flask to accept connections from outside the container (not just from inside it). Without this, your browser on Windows could not reach the app running inside Docker.

File 5a of 5 — Dockerfile

The Dockerfile is a recipe that tells Docker how to build your application into a container image. Read each line's comment — understanding this file is important for the whole project.

```
# Use Python 3.11 slim — a minimal Linux image with Python pre-installed
FROM python:3.11-slim

# Set the working directory inside the container
# All subsequent commands run from this folder
WORKDIR /app

# Copy requirements.txt first (before the rest of the code)
# Docker caches this layer — if requirements.txt hasn't changed,
# it skips the pip install step on rebuilds (saves time)
COPY requirements.txt .

# Install all Python libraries listed in requirements.txt
# --no-cache-dir keeps the image smaller
RUN pip install --no-cache-dir -r requirements.txt

# Copy all project files into the container's /app folder
COPY . .

# Tell Docker that the app listens on port 5000
# (this is documentation only — the actual port mapping is in docker-
compose.yml)
EXPOSE 5000

# The command that runs when the container starts
CMD ["python", "app.py"]
```

File 5b of 5 — docker-compose.yml

docker-compose.yml defines your app as a service and lets you start everything with one command. It also handles port mapping (connecting Docker's port 5000 to your Windows port 5000) and loads your .env file automatically.

```yaml
version: '3.9'

services:
  web:
    # Build the image from the Dockerfile in the current folder
    build: .

    # Name the container for easy reference
    container_name: ems-web

    # Map port 5000 on your Windows machine to port 5000 in the container
    # Format: host_port:container_port
    ports:
      - "5000:5000"

    # Load environment variables from your .env file
    env_file:
      - .env

    # Mount your project folder into the container
    # Changes you make to files are reflected instantly (no rebuild needed)
    volumes:
      - .:/app

    # Restart the container automatically if it crashes
    restart: unless-stopped
```

Phase 5 of 6

Build & Run the Docker Container

Before running Docker — create your .env file:

Docker needs a .env file to load environment variables. We will create one now from the example template. In Git Bash:

```
cp .env.example .env
```

This creates a copy of .env.example named .env. For today, the placeholder values are fine — the Flask app does not connect to Firebase or PayPal yet.

> **Important — .env must never go to GitHub**
> The .gitignore file GitHub created already includes .env. Double-check by opening .gitignore in VS Code and confirming you see a line that says .env in it. If it is not there, add it manually on a new line.

Build the Docker image:

This command reads your Dockerfile and builds a container image. It downloads Python, installs all your libraries, and packages everything. The first time takes 2–4 minutes depending on your internet speed.

```
docker compose build
```

You will see a lot of output scrolling past. This is Docker downloading and installing things. Wait for it to finish and return to the command prompt.

> **If you see an error during build**
> The most common error is failed to solve: python:3.11-slim: not found — this means Docker cannot connect to the internet. Check that Docker Desktop is running (Engine running status at the bottom left) and try again.

Start the container:

```
docker compose up
```

You will see output like this — this means your Flask app is running inside Docker:

```
ems-web  | * Running on all addresses (0.0.0.0)
ems-web  | * Running on http://127.0.0.1:5000
```

```
ems-web  | * Running on http://172.18.0.2:5000
ems-web  | Press CTRL+C to quit
```

Verify in the browser:

Open your browser (Chrome or Edge) and go to:

```
http://localhost:5000
```

You should see this JSON response in the browser:

```
{
  "message": "Hello from EMS! Docker is working.",
  "status":  "ok",
  "project": "event-management-system",
  "day":     1,
  "sprint":  "Sprint 1 — Foundation"
}
```

> **Day 1 core goal achieved!**
> If you see this response in your browser, your Docker container is running Flask successfully. This is your end-of-day deliverable.

Stop the container:

Press `Ctrl + C` in Git Bash to stop the running container. To start it again later, run `docker compose up` from the project folder.

Phase 6 of 6

Commit Everything to GitHub

Check what Git sees:

In Git Bash (make sure Docker is stopped first with Ctrl+C), run:

```
git status
```

Git will list all new files in red under Untracked files. You should see all the files we created today. The .env file should NOT appear — if it does, your .gitignore is not set up correctly (see the note in Phase 5).

Stage all files:

Staging means telling Git which files to include in the next snapshot (commit):

```
git add .
```

Run git status again. All files should now appear in green under Changes to be committed.

Commit with a message:

A commit is a permanent snapshot of your project at this moment. The message describes what you did:

```
git commit -m "Day 1: project scaffold, Docker setup, Flask hello-world"
```

You should see output confirming the commit, with a list of all the files included.

Push to GitHub:

This uploads your commit to GitHub so it is safely backed up online:

```
git push origin develop
```

If Git asks for your GitHub username and password: use your GitHub username, and for the password use a Personal Access Token (not your GitHub password). To create one: GitHub → Settings → Developer Settings → Personal Access Tokens → Generate new token → tick repo → copy the token.

Verify on GitHub

Open https://github.com/YOUR-USERNAME/event-management-system in your browser. Switch to the develop branch using the branch dropdown. You should see all your files there.

End-of-Day 1 Checklist

Go through each item below. Every box should be ticked before you finish for the day.

	Item	How to verify
☐	**Git installed**	Run `git --version` in Git Bash — shows a version number
☐	**Python installed**	Run `python --version` in Git Bash — shows Python 3.11.x
☐	**Docker installed**	Run `docker --version` in Git Bash — shows a version number
☐	**Docker running**	Docker Desktop shows `Engine running` at the bottom left
☐	**Repo cloned**	The event-management-system folder exists in Documents
☐	**develop branch active**	Git Bash prompt shows `(develop)` after the folder name
☐	**Folder structure**	VS Code Explorer shows all 5 folders and 5+ files
☐	**app.py written**	File contains the Flask route that returns the JSON message
☐	**Dockerfile written**	File contains FROM python:3.11-slim and the CMD instruction
☐	**docker-compose.yml**	File contains ports: 5000:5000 and env_file: .env
☐	**.env.example written**	File contains FLASK_ENV, FIREBASE_*, PAYPAL_*, SENDGRID_* keys
☐	**requirements.txt**	File lists Flask, firebase-admin, paypalrestsdk, sendgrid, qrcode
☐	**.env created**	Copy of .env.example exists as .env (NOT in Git)
☐	**Docker builds**	`docker compose build` completes without errors
☐	**Container runs**	`docker compose up` starts and shows Running on 0.0.0.0:5000
☐	**Browser test**	http://localhost:5000 shows the Hello from EMS! JSON response
☐	**Committed to Git**	`git log` shows the Day 1 commit
☐	**Pushed to GitHub**	GitHub repo's develop branch shows all today's files

What's Next — Day 2 Preview

Tomorrow (Day 2) you will connect your project to Firebase. Specifically:

- Create a Firebase project in the Firebase Console
- Enable Firestore database and Firebase Authentication
- Download the service account JSON key and add it to Docker
- Install and configure the firebase-admin SDK inside Flask
- Write a smoke test: create and read a document in Firestore

By end of Day 2 your Flask app will be talking to a real Firebase database. No frontend yet — just the backend connection confirmed and tested.

> **To prepare for Day 2**
> Create your Firebase account now if you don't have one: https://firebase.google.com — sign in with your Google account. It's free for the usage levels in this project.

Day 2 — Setup Guide

Firebase Setup & Firestore Schema

event-management-system | Sprint 1 | 5 Parts | ~4 hours

Part A	Fix Day 1 — restructure templates/ and static/ folders (~10 min)
Part B	Create Firebase project, enable Firestore + Auth, download key (~20 min)
Part C	Write firebase_config.py, wire into Docker and Flask (~45 min)
Part D	Design full Firestore schema from all 20 wireframe pages (~45 min)
Part E	Write /test-firebase route, rebuild Docker, verify in browser (~20 min)
End goal	Browser confirms: Firebase connected — Firestore read/write working

Part A — Fix from Day 1

Restructure templates/ and static/ folders

Your wireframes have 20 pages across 4 user roles (Public, Attendee, Organizer, Admin). Yesterday we created a flat templates/ folder. With 20 HTML files that will become impossible to manage. We fix this now before adding any more code.

The new folder structure:

```
event-management-system/
├── app/
│   └── __init__.py
├── templates/
│   ├── public/          ← pages 1, 2 (event listing, event detail)
│   ├── auth/            ← login, signup, forgot password
│   ├── attendee/        ← pages 10, 11 (my events, ticket view)
│   ├── organizer/       ← pages 6-9, 12-17 (dashboard, manage)
│   ├── admin/           ← page 20 (user management)
│   └── base.html        ← shared layout all pages will extend
├── static/
│   ├── css/             ← Bootstrap + custom styles
│   ├── js/              ← custom JavaScript files
│   └── images/          ← logos, icons, event cover images
├── secrets/             ← Firebase key lives here (NEVER in Git)
│   └── firebase-key.json
├── app.py
├── requirements.txt
├── Dockerfile
├── docker-compose.yml
├── .env.example
├── .env
└── .gitignore
```

Run these commands in Git Bash:

Open Git Bash, navigate to your project, then run:

```
cd ~/Documents/event-management-system

# Create template subfolders
mkdir -p templates/public templates/auth templates/attendee
templates/organizer templates/admin

# Create static subfolders
mkdir -p static/css static/js static/images

# Create the secrets folder (for Firebase key)
mkdir secrets
```

```
# Create placeholder files so Git tracks the empty folders
touch templates/public/.gitkeep templates/auth/.gitkeep
touch templates/attendee/.gitkeep templates/organizer/.gitkeep
templates/admin/.gitkeep
touch static/css/.gitkeep static/js/.gitkeep static/images/.gitkeep
touch templates/base.html
```

CRITICAL — add secrets/ to .gitignore

The secrets/ folder will contain your Firebase private key. It must NEVER be pushed to GitHub. Open .gitignore in VS Code and add these two lines at the bottom: secrets/ .env Save the file. Run git status and confirm secrets/ does not appear in the list.

Commit this structure:

```
git add .
git commit -m "Day 2A: restructure templates/ and static/ for 20-page
wireframe"
```

Part B — Firebase Console

Create Project, Enable Firestore & Auth, Download Key

Step 1/5

Create the Firebase project

- Open your browser and go to: https://console.firebase.google.com
- Sign in with your Google account
- Click the "Add project" card (the one with the + icon)
- Project name: type ems-arfath (or any name you like — this is just the display name)
- Google Analytics: click Continue (leave it enabled — it's free and harmless)
- Analytics account: choose Default Account for Firebase
- Click "Create project" — wait about 30 seconds for it to finish
- Click "Continue" when the success screen appears

Step 2/5

Enable Firestore Database

- In the left sidebar, click Build → Firestore Database
- Click "Create database"
- On the security rules screen, select: Start in test mode
  - This allows all reads and writes for 30 days — safe for development
  - We will lock it down with proper rules in Sprint 8
- Click "Next"
- Location: choose asia-south1 (Mumbai) — closest to Udupi, gives lowest latency
- Click "Enable" — wait about 20 seconds

You will see the Firestore console

It shows an empty database with a Start collection button. Leave it empty for now — we will create collections from our Flask code.

Step 3/5

Enable Firebase Authentication

- In the left sidebar, click Build → Authentication
- Click "Get started"
- You will see the Sign-in providers screen
- Click "Email/Password" (first option in the list)
- Toggle Enable to ON (the first toggle only — leave Email link disabled)
- Click "Save"

That's all for Auth today

Email/Password auth is all we need for MVP v1. Google Sign-In and other providers can be added later if you want.

Step 4/5 Enable Firebase Storage

- In the left sidebar, click Build → Storage
- Click "Get started"
- Security rules: select Start in test mode
- Click "Next" → location is already set to asia-south1 → click "Done"

Storage is needed for: event cover images (page 7), QR code PNGs (page 5, 11), attendance certificates (post-event), and speaker photos (page 14).

Step 5/5 Generate and download the service account key

The service account key is a JSON file that lets your Flask backend prove to Firebase that it is authorised to read and write data. This is different from the frontend Firebase config — it has full admin access, which is why it must never go to GitHub.

- In the Firebase Console, click the gear icon (top left, next to Project Overview)
- Select "Project settings"
- Click the "Service accounts" tab
- You will see the Firebase Admin SDK section
- Make sure Python is selected in the code snippet (not Node.js)
- Click the blue "Generate new private key" button
- Click "Generate key" on the confirmation dialog
- A JSON file downloads to your Windows Downloads folder — it has a long name like ems-arfath-firebase-adminsdk-xxxxx-xxxxxxxxxx.json

Move the key into your project:

In Git Bash:

```
# Move the downloaded key into the secrets/ folder
# Replace the filename below with your actual downloaded filename
mv "/mnt/c/Users/YOUR-WINDOWS-USERNAME/Downloads/ems-arfath-firebase-
adminsdk-*.json"
   secrets/firebase-key.json
```

How to find your Windows username

In Git Bash, run: echo $USERNAME — it will print your Windows username. Use that in the path above instead of YOUR-WINDOWS-USERNAME. Alternatively: open File Explorer, go to Downloads, and drag the file into the secrets/ folder in VS Code's Explorer panel.

Verify the file is in place:

```
ls secrets/
# Should print:  firebase-key.json
```

Verify Git is ignoring it:

```
git status
# secrets/ should NOT appear in the output
# If it does appear, fix your .gitignore before continuing
```

Part C — Flask Integration

Write firebase_config.py and wire into Docker

Step 1 — Update your .env file

Open .env in VS Code and add the Firebase values. You need three pieces of information from your Firebase project:

Variable	Where to find it	Example value
FIREBASE_PROJECT_ID	Firebase Console → Project Settings → General → Project ID	ems-arfath
FIREBASE_STORAGE_BUCKET	Firebase Console → Storage → copy the bucket name shown	ems-arfath.appspot.com
FIREBASE_CREDENTIALS_PATH	Fixed path — we always put it here	secrets/firebase-key.json

Your .env file should now look like this (replace the placeholder values with your actual ones):

```
# ─── Flask ──────────────────────────────────────────
FLASK_ENV=development
FLASK_DEBUG=1
SECRET_KEY=change-this-to-a-random-string-32-chars

# ─── Firebase ───────────────────────────────────────
FIREBASE_CREDENTIALS_PATH=secrets/firebase-key.json
FIREBASE_PROJECT_ID=ems-arfath
FIREBASE_STORAGE_BUCKET=ems-arfath.appspot.com

# ─── PayPal (sandbox — fill in on Day 10) ──────────
PAYPAL_MODE=sandbox
PAYPAL_CLIENT_ID=your-paypal-sandbox-client-id
PAYPAL_CLIENT_SECRET=your-paypal-sandbox-client-secret

# ─── SendGrid (fill in on Day 12) ──────────────────
SENDGRID_API_KEY=your-sendgrid-api-key
SENDGRID_FROM_EMAIL=noreply@yourdomain.com

# ─── QR Code ────────────────────────────────────────
QR_HMAC_SECRET=another-random-string-change-this
```

Step 2 — Update docker-compose.yml to mount the secrets folder

Right now Docker can see your project files but not the secrets/ folder, because the JSON key needs to be explicitly accessible inside the container. Open docker-compose.yml in VS Code and replace the entire file with this:

```yaml
version: '3.9'

services:
  web:
    build: .
    container_name: ems-web

    ports:
      - "5000:5000"

    env_file:
      - .env

    volumes:
      # Mount entire project (live reload — changes appear instantly)
      - .:/app
      # Mount secrets folder explicitly so Flask can read firebase-key.json
      - ./secrets:/app/secrets:ro

    restart: unless-stopped
```

> **What :ro means**
> The :ro at the end of the secrets volume mount means read-only. The container can read the key file but cannot modify or delete it. This is a security best practice.

Step 3 — Write app/firebase_config.py

Create a new file: app/firebase_config.py in VS Code. This module handles the Firebase connection and exposes three objects your routes will use: db (Firestore), auth (Firebase Auth), and bucket (Firebase Storage).

```python
import os
import firebase_admin
from firebase_admin import credentials, firestore, auth, storage

# ── Module-level singletons ──────────────────────────────────────────
# These are set once when init_firebase() is called and reused everywhere
_app  = None
db    = None
bucket= None


def init_firebase():
    """
    Initialise Firebase Admin SDK.
    Called once from app.py at startup.
    Safe to call multiple times — returns early if already initialised.
    """
    global _app, db, bucket

    # Guard: do not initialise twice (e.g. during Flask auto-reload)
    if firebase_admin._apps:
        db     = firestore.client()
        bucket = storage.bucket()
```

```
        return

    # Load credentials from the JSON key file
    key_path = os.getenv('FIREBASE_CREDENTIALS_PATH', 'secrets/firebase-
key.json')
    cred = credentials.Certificate(key_path)

    # Initialise the app with project ID and storage bucket
    _app = firebase_admin.initialize_app(cred, {
        'projectId':     os.getenv('FIREBASE_PROJECT_ID'),
        'storageBucket': os.getenv('FIREBASE_STORAGE_BUCKET')
    })

    # Create module-level client objects
    db     = firestore.client()
    bucket = storage.bucket()

    print('[Firebase] Initialised successfully',
          f'Project: {os.getenv("FIREBASE_PROJECT_ID")}')
```

Step 4 — Update app.py to call init_firebase()

Open app.py in VS Code and replace the entire file with this updated version. The only additions are: importing init_firebase, calling it at startup, and adding the test route.

```
import os
from flask import Flask, jsonify
from dotenv import load_dotenv
from app.firebase_config import init_firebase

load_dotenv()

app = Flask(__name__)
app.config['SECRET_KEY'] = os.getenv('SECRET_KEY', 'dev-secret')

# Initialise Firebase when the app starts
init_firebase()

# ── Routes ──────────────────────────────────────────────────────────
from app.routes.test import test_bp
app.register_blueprint(test_bp)

@app.route('/')
def index():
    return jsonify({
        'message': 'EMS running. Visit /test-firebase to verify DB.',
        'status':  'ok',
        'day':     2
    })

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```

Step 5 — Create app/routes/ folder and test route

We will keep all routes in separate files inside app/routes/. This is the blueprint pattern — it keeps app.py clean as the project grows to 20+ pages.

In Git Bash:

```
mkdir -p app/routes
touch app/routes/__init__.py app/routes/test.py
```

Now open app/routes/test.py in VS Code and paste this:

```
from flask import Blueprint, jsonify
from app.firebase_config import db
from datetime import datetime, timezone

test_bp = Blueprint('test', __name__)


@test_bp.route('/test-firebase')
def test_firebase():
    """
    Smoke test: write a doc to Firestore, read it back, delete it.
    Returns JSON confirming each step worked.
    Remove this route before going to production.
    """
    results = {}

    try:
        # ── 1. Write ──────────────────────────────────────────────
        test_ref = db.collection('_smoke_test').document('day2')
        test_ref.set({
            'project':    'event-management-system',
            'test_day':   2,
            'written_at': datetime.now(timezone.utc).isoformat()
        })
        results['write'] = 'ok'

        # ── 2. Read back ──────────────────────────────────────────
        snap = test_ref.get()
        data = snap.to_dict()
        results['read']  = 'ok' if data else 'empty'

        # ── 3. Delete ─────────────────────────────────────────────
        test_ref.delete()
        results['delete'] = 'ok'

        return jsonify({
            'status':   'success',
            'message': 'Firebase connected — Firestore read/write working.',
            'results': results
        }), 200

    except Exception as e:
        return jsonify({
            'status':  'error',
            'message':str(e)
        }), 500
```

Part D — Firestore Schema

Full Data Model from All 20 Wireframe Pages

This is the complete Firestore schema designed from your 20 wireframe pages. Every field is mapped to the page(s) that need it. You do not create these collections manually — Flask will create them automatically when it first writes a document. This section is your reference for the rest of the project.

> **How Firestore works (quick recap)**
> Firestore is a document database. Collections are like table names. Documents are like rows. Each document is a JSON object with fields. Unlike SQL, there is no fixed schema — but we plan it carefully here so our code is consistent. Subcollections (e.g. ticket_types inside events) are used when data belongs exclusively to a parent.

Collection 1 — users

Path: /users/{uid} | uid = Firebase Auth user ID | Used by: pages 1, 3, 10, 11, 18, 20

Field	Type	Req?	Pages	Notes
uid	string	**Yes**	All	Firebase Auth UID — matches the document ID
email	string	**Yes**	3,10,20	Login email — unique across all users
full_name	string	**Yes**	3,5,9,10	Display name on tickets and certificates
phone	string	No	3	Optional — collected at registration
company	string	No	3,18	Optional — shown in attendee directory pg.18
job_title	string	No	18	Shown in attendee directory pg.18
profile_picture_url	string	No	10	Firebase Storage URL — profile avatar
role	string	**Yes**	20	Values: attendee | organizer | admin
is_active	boolean	**Yes**	20	False = suspended (admin action pg.20)
networking_opt_in	boolean	**Yes**	18	True = visible in attendee directory pg.18
created_at	timestamp	**Yes**	20	Server timestamp — set on signup
last_login_at	timestamp	No	20	Updated on each login

Collection 2 — venues

Path: /venues/{venue_id} | Used by: pages 7, 1, 2

Field	Type	Req?	Pages	Notes
venue_id	string	**Yes**	7	Auto-generated Firestore document ID

name	string	**Yes**	1,2,7	e.g. City Hall, San Francisco
address	string	**Yes**	7	Street address
city	string	**Yes**	1,2	Used on event listing cards pg.1
capacity	number	**Yes**	7	Maximum attendee count
contact_name	string	No	7	Venue contact person
contact_phone	string	No	7	Venue contact phone
floor_plan_url	string	No	7	Firebase Storage URL (Sprint 3+)
organizer_uid	string	**Yes**	7	Owner — only this organizer can edit
created_at	timestamp	**Yes**	7	Server timestamp

Collection 3 — events

Path: /events/{event_id} | Used by: pages 1, 2, 6, 7, 8, 9, 12, 13, 17

Field	Type	Req?	Pages	Notes
event_id	string	**Yes**	All	Auto-generated Firestore document ID
organizer_uid	string	**Yes**	6,7	Owner — only this user can manage
name	string	**Yes**	1,2,6,7	e.g. TechConf 2026
description	string	**Yes**	2,7	Full event description
start_datetime	timestamp	**Yes**	1,2,6,7,13	Event start — date + time
end_datetime	timestamp	**Yes**	7,13	Event end — date + time
venue_id	string	**Yes**	2,7	Reference to /venues/{venue_id}
event_type	string	**Yes**	7	Values: physical | virtual | hybrid
status	string	**Yes**	6,7	Values: draft | published | cancelled | completed
cover_image_url	string	No	1,2,6	Firebase Storage URL — event banner image
branding_color	string	No	7	Hex color e.g. #185FA5 for event page styling
streaming_url	string	No	10	YouTube/Zoom link for virtual events pg.10
custom_fields	array	No	16	List of field config objects from pg.16 builder
total_registrations	number	**Yes**	6,17	Maintained by Firestore atomic increment
total_revenue	number	**Yes**	6,17	Sum of confirmed payment amounts
total_checkins	number	**Yes**	6,12,17	Maintained by Firestore atomic increment
created_at	timestamp	**Yes**	6	Server timestamp
published_at	timestamp	No	6	Set when status changes to published

Subcollection: ticket_types

Ticket types live at /events/{event_id}/ticket_types/{ticket_type_id} — they belong exclusively to their event so a subcollection is cleaner than a top-level collection. See page 8 of your wireframes.

Subcollection: ticket_types	Type	Req?	Pages	Notes
ticket_type_id	string	**Yes**	8	Auto-generated document ID
name	string	**Yes**	2,8	e.g. Early Bird | General Admission | VIP
description	string	No	2	Short description shown on event detail pg.2
price	number	**Yes**	2,3,4,8	In INR. 0 = free ticket
quantity_total	number	**Yes**	2,8	Total tickets available
quantity_sold	number	**Yes**	2,6,8,17	Atomically incremented on each purchase
max_per_order	number	**Yes**	2,8	Max a single attendee can buy e.g. 4
sales_start	timestamp	No	8	If set, ticket not available before this time
sales_end	timestamp	No	8	If set, ticket not available after this time
is_active	boolean	**Yes**	8	False = hidden from registration page

Collection 4 — registrations

Path: /registrations/{registration_id} | Used by: pages 3, 4, 5, 9, 10, 11, 12

Field	Type	Req?	Pages	Notes
registration_id	string	**Yes**	5,9,11	Auto-generated — also used as order number
order_number	string	**Yes**	5,11	Human-readable: ORD-XXXXX from registration_id prefix
event_id	string	**Yes**	All	Reference to /events/{event_id}
attendee_uid	string	**Yes**	10,11	Reference to /users/{uid}
ticket_type_id	string	**Yes**	5,9,11	Reference to ticket_types subcollection
ticket_type_name	string	**Yes**	5,11	Denormalised — stored for fast display without join
quantity	number	**Yes**	3,4	Number of tickets purchased in this order
unit_price	number	**Yes**	4,9	Price per ticket at time of purchase
subtotal	number	**Yes**	4	unit_price x quantity
discount_amount	number	**Yes**	4	0 if no promo code applied
total_amount	number	**Yes**	4,9	Amount actually charged
promo_code_used	string	No	4	The code string e.g. SAVE10

status	string	**Yes**	9,12	Values: pending | confirmed | cancelled | checked_in
qr_code_url	string	**Yes**	5,11,12	Firebase Storage URL of QR code PNG
qr_payload	string	**Yes**	12	HMAC-signed string embedded in the QR code
custom_field_answers	map	No	9	Key-value pairs from custom registration fields pg.16
paypal_order_id	string	No	4	PayPal order ID from create-order response
paypal_transaction_id	string	No	4	PayPal capture transaction ID — for records
payment_status	string	**Yes**	9	Values: pending | paid | refunded | failed
email_sent	boolean	**Yes**	5	True once confirmation email delivered
checked_in_at	timestamp	No	12	Server timestamp set at check-in pg.12
checked_in_by	string	No	12	UID of staff who scanned the QR pg.12
certificate_url	string	No	10	Firebase Storage URL — post-event certificate
created_at	timestamp	**Yes**	9	Server timestamp — registration created
confirmed_at	timestamp	No	9	Server timestamp — payment confirmed

Collection 5 — promo_codes

Path: /promo_codes/{code_id} | Used by: pages 2, 3, 4, 15

Field	Type	Req?	Pages	Notes
code_id	string	**Yes**	15	Auto-generated document ID
event_id	string	**Yes**	15	Promo codes are scoped to a specific event
code	string	**Yes**	2,3,4,15	The code string e.g. SAVE10 — stored uppercase
discount_type	string	**Yes**	15	Values: percentage | fixed
discount_value	number	**Yes**	3,4,15	Percent (10) or fixed amount (20)
valid_from	timestamp	**Yes**	15	Code not usable before this datetime
valid_until	timestamp	**Yes**	15	Code not usable after this datetime
max_uses	number	**Yes**	15	Total redemptions allowed e.g. 100
used_count	number	**Yes**	15	Atomically incremented on each use
is_active	boolean	**Yes**	15	False = code manually deactivated by organizer
created_at	timestamp	**Yes**	15	Server timestamp

Collection 6 — sessions

Path: /sessions/{session_id} | Used by: pages 10, 13, 14, 17

Field	Type	Req?	Pages	Notes
session_id	string	**Yes**	13	Auto-generated document ID
event_id	string	**Yes**	13	Parent event reference
title	string	**Yes**	13	Session title e.g. AI/ML Workshop
description	string	No	13	Full session description
track	string	No	13	e.g. Track A | Track B — for multi-track schedule pg.13
location	string	No	13	Room or stage name within the venue
start_datetime	timestamp	**Yes**	13	Session start
end_datetime	timestamp	**Yes**	13	Session end
speaker_uids	array	No	13,14	List of speaker document IDs
streaming_url	string	No	10,13	YouTube/Zoom embed URL for virtual sessions
session_day	number	**Yes**	13	Day number e.g. 1 for Day 1 — for agenda grouping
saved_by_uids	array	No	10	List of attendee UIDs who saved this session pg.10
attendee_count	number	No	17	Derived count for analytics pg.17

Collection 7 — speakers

Speakers live at /speakers/{speaker_id} with fields: event_id, name, email, bio, photo_url, social_links (map), sessions (array of session_ids). Created from wireframe page 14.

Part E — Build & Verify

Rebuild Docker and test the Firebase connection

Step 1 — Rebuild the Docker image

We changed requirements.txt (firebase-admin was already listed) and added new Python files. We need to rebuild the image so Docker picks up the new files.

In Git Bash (make sure you are in the project folder):

```
docker compose down
docker compose build --no-cache
```

> **Why --no-cache?**
> Docker caches each build step. The --no-cache flag forces it to redo every step from scratch — this guarantees it picks up the new files and installs the latest libraries. Only use it when you have made significant changes. Normal day-to-day rebuilds can use just docker compose build.

Step 2 — Start the container

```
docker compose up
```

Watch the startup output carefully. You should see this line printed by firebase_config.py:

```
ems-web  | [Firebase] Initialised successfully  Project: ems-arfath
```

> **If you see a FileNotFoundError**
> It means Docker cannot find secrets/firebase-key.json. Check that: 1. The file exists at secrets/firebase-key.json in your project folder 2. The volume mount in docker-compose.yml shows: - ./secrets:/app/secrets:ro 3. Run docker compose down then docker compose up again.

Step 3 — Test the Firebase connection in the browser

Open your browser and go to:

```
http://localhost:5000/test-firebase
```

You should see this JSON response — all three values must be ok:

```
{
  "status":   "success",
  "message":  "Firebase connected — Firestore read/write working.",
  "results": {
```

```
    "write":  "ok",
    "read":   "ok",
    "delete": "ok"
  }
}
```

> **What just happened**
> Flask wrote a document to a collection called _smoke_test in your Firestore database, read it back, and deleted it. You can verify in the Firebase Console (Build → Firestore) — the collection will not be there because we deleted it. That is correct behaviour.

Step 4 — Commit everything

```
git add .
git commit -m "Day 2: Firebase setup, Firestore schema design, test route
working"
git push origin develop
```

End-of-Day 2 Checklist

Every box should be ticked before you finish. If any step failed, scroll back to that part of the guide.

	Item	How to verify
☐	**templates/ restructured**	VS Code Explorer shows auth/, public/, attendee/, organizer/, admin/ subfolders
☐	**static/ restructured**	VS Code Explorer shows static/css/, static/js/, static/images/ subfolders
☐	**secrets/ folder created**	The secrets/ folder exists at the project root
☐	**secrets/ in .gitignore**	Running `git status` does NOT show secrets/ in the output
☐	**Firebase project created**	Firebase Console shows your project at console.firebase.google.com
☐	**Firestore enabled**	Firebase Console → Build → Firestore shows the database (empty)
☐	**Auth enabled**	Firebase Console → Build → Auth → Sign-in providers shows Email/Password ON
☐	**Storage enabled**	Firebase Console → Build → Storage shows the bucket
☐	**Key downloaded**	secrets/firebase-key.json exists and contains JSON starting with {
☐	**.env updated**	.env has real values for FIREBASE_PROJECT_ID and FIREBASE_STORAGE_BUCKET
☐	**docker-compose updated**	docker-compose.yml has the `./secrets:/app/secrets:ro` volume line
☐	**firebase_config.py written**	app/firebase_config.py exists with init_firebase() function
☐	**app/routes/ created**	app/routes/__init__.py and app/routes/test.py both exist
☐	**Docker rebuilt**	docker compose build --no-cache completed without errors
☐	**Firebase init log line**	docker compose up shows: [Firebase] Initialised successfully
☐	**Smoke test passes**	http://localhost:5000/test-firebase returns all three results as ok
☐	**Committed & pushed**	GitHub develop branch shows the Day 2 commit

What's Next — Day 3 Preview

Tomorrow is the first day of real feature code. You will build the complete authentication system — sign up, log in, log out, password reset, and role-based route guards — using Firebase Auth + Flask sessions.

- Implement POST /auth/signup — create user in Firebase Auth and write profile to /users/ in Firestore
- Implement POST /auth/login — verify Firebase ID token, set Flask session cookie
- Implement POST /auth/logout — clear session
- Implement POST /auth/forgot-password — trigger Firebase password reset email
- Write login_required and role_required(role) decorators
- Create the auth/ HTML templates: signup.html, login.html, forgot_password.html using base.html

Prepare for Day 3

The templates you build on Day 3 will extend base.html — the shared layout file we created today (templates/base.html is currently empty). If you have time tonight, you can write the Bootstrap 5 base layout with navbar and flash message support. This is optional — we will write it on Day 3 if not done.

Day 3 — Setup Guide

Authentication System

Flask + Firebase Auth + Bootstrap 5 | 5 Parts | ~6 hours

Part A	Write base.html — master layout for all 20 pages (~45 min)
Part B	Write 3 auth HTML templates — signup, login, forgot password (~45 min)
Part C	Write app/routes/auth.py — all 4 auth routes (~90 min)
Part D	Write app/decorators.py — login_required & role_required (~30 min)
Part E	Wire up, rebuild Docker, test all flows in browser (~30 min)
End goal	Sign up, log in, log out, reset password all working in browser

Overview — How authentication works in this app

Firebase Auth handles the identity layer — it stores passwords securely and issues ID tokens. Flask handles the session layer — it stores the user's UID in an encrypted cookie so they stay logged in between page loads. Firestore stores the user's profile (name, role, etc.).

The full flow looks like this:

#	Action	What happens
1	**User submits signup form**	Flask calls Firebase Auth REST API → creates user → saves profile to Firestore /users/ → sets session cookie → redirects to dashboard
2	**User submits login form**	Flask calls Firebase Auth REST API → verifies password → gets ID token → validates token with firebase-admin → sets session cookie
3	**User visits protected page**	login_required decorator checks session → if no session, redirect to /auth/login
4	**Organizer visits organizer page**	role_required('organizer') checks session role → if attendee or admin, redirect with error
5	**User clicks logout**	Flask clears the session cookie → redirect to home page
6	**User clicks forgot password**	Flask calls Firebase Auth REST API → Firebase sends reset email directly to user

> **Why Firebase Auth REST API for login/signup?**
> The firebase-admin SDK (installed on Day 2) is for server-side admin operations — it cannot verify a password. For login and signup we use the Firebase Auth REST API (a simple HTTP call with your Web API key). The admin SDK is then used to verify the ID token that comes back.

New file created today

```
event-management-system/
├── app/
│   ├── firebase_config.py   (Day 2 — unchanged)
│   ├── decorators.py        ← NEW — login_required, role_required
│   └── routes/
│       ├── test.py          (Day 2 — unchanged)
│       └── auth.py          ← NEW — signup, login, logout, forgot-
password
├── templates/
│   ├── base.html            ← NEW — master layout
│   └── auth/
│       ├── signup.html      ← NEW
│       ├── login.html       ← NEW
│       └── forgot_password.html ← NEW
├── static/css/
│   └── ems.css             ← NEW — custom styles
└── app.py                   (updated — register auth blueprint)
```

Part A

Write base.html — The Master Layout

base.html is the single file that defines the navbar, page structure, and footer that all 20 pages share. Every other HTML file will extend this one using Jinja2 template inheritance. You write the layout once here and never repeat it.

Step 1 — Create static/css/ems.css

Create a new file at static/css/ems.css in VS Code and paste this:

```
/* ── EMS Custom Styles ──────────────────────────────────────────── */
:root {
  --ems-primary:    #0d6efd;   /* Bootstrap blue */
  --ems-primary-dk: #0a58ca;   /* Hover state */
  --ems-navbar-bg:  #0d47a1;   /* Deep blue navbar */
  --ems-footer-bg:  #1a237e;   /* Darker blue footer */
  --ems-card-shadow: 0 2px 8px rgba(0,0,0,0.10);
}

/* Navbar */
.ems-navbar { background-color: var(--ems-navbar-bg) !important; }
.ems-navbar .navbar-brand { font-weight: 700; font-size: 1.4rem; color: #fff
!important; }
.ems-navbar .nav-link { color: rgba(255,255,255,0.85) !important; }
.ems-navbar .nav-link:hover { color: #fff !important; }

/* Cards */
.ems-card { box-shadow: var(--ems-card-shadow); border: none; border-radius:
10px; }

/* Auth pages — centered card */
.auth-wrapper { min-height: 80vh; display: flex; align-items: center;
justify-content: center; }
.auth-card { width: 100%; max-width: 440px; }

/* Footer */
.ems-footer { background-color: var(--ems-footer-bg); color:
rgba(255,255,255,0.75); padding: 1.5rem 0; margin-top: auto; }
.ems-footer a { color: rgba(255,255,255,0.75); text-decoration: none; }
.ems-footer a:hover { color: #fff; }

/* Body — flex column so footer sticks to bottom */
body { display: flex; flex-direction: column; min-height: 100vh; }
main { flex: 1; }

/* Flash messages */
.flash-container { position: fixed; top: 70px; right: 20px; z-index: 9999;
min-width: 300px; }
```

Step 2 — Write templates/base.html

Create templates/base.html in VS Code. This is the most important file you write today — every page inherits from it. Read each comment carefully.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  {# ── Page title: child templates override this block ── #}
  <title>{% block title %}EMS{% endblock %} | Event Management System</title>

  {# ── Bootstrap 5 CSS (CDN) ── #}
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css">

  {# ── Bootstrap Icons ── #}
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-
icons@1.11.3/font/bootstrap-icons.min.css">

  {# ── Custom EMS styles ── #}
  <link rel="stylesheet" href="{{ url_for('static', filename='css/ems.css') }}">

  {# ── Extra head content (charts, etc.) from child pages ── #}
  {% block extra_head %}{% endblock %}
</head>
<body>

  {# ═══ NAVBAR ═══════════════════════════════════════════════ #}
  <nav class="navbar navbar-expand-lg ems-navbar">
    <div class="container">
      {# Brand / Logo #}
      <a class="navbar-brand" href="{{ url_for('public.index') }}">
        <i class="bi bi-calendar-event me-2"></i>EMS
      </a>

      {# Mobile hamburger button #}
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-
bs-target="#navMenu">
        <span class="navbar-toggler-icon"></span>
      </button>

      <div class="collapse navbar-collapse" id="navMenu">
        <ul class="navbar-nav ms-auto align-items-lg-center">

          {# Show different links based on login state and role #}
          {% if session.get('uid') %}

            {# Organizer links #}
            {% if session.get('role') == 'organizer' %}
            <li class="nav-item">
              <a class="nav-link" href="{{ url_for('organizer.dashboard') }}">
                <i class="bi bi-speedometer2 me-1"></i>Dashboard
              </a></li>
            {% endif %}

            {# Admin links #}
            {% if session.get('role') == 'admin' %}
```

```
            <li class="nav-item">
              <a class="nav-link" href="{{ url_for('admin.dashboard') }}">
                <i class="bi bi-shield-check me-1"></i>Admin
              </a></li>
            {% endif %}

            {# Attendee / all logged-in users #}
            <li class="nav-item">
              <a class="nav-link" href="{{ url_for('attendee.my_events') }}">
                <i class="bi bi-ticket-perforated me-1"></i>My Events
              </a></li>

            {# User dropdown #}
            <li class="nav-item dropdown">
              <a class="nav-link dropdown-toggle" href="#" data-bs-
toggle="dropdown">
                <i class="bi bi-person-circle me-1"></i>{{ session.get('email',
'Account') }}
              </a>
              <ul class="dropdown-menu dropdown-menu-end">
                <li><a class="dropdown-item" href="{{ url_for('auth.logout') }}">
                  <i class="bi bi-box-arrow-right me-2"></i>Logout</a></li>
              </ul>
            </li>

          {% else %}
            {# Not logged in #}
            <li class="nav-item"><a class="nav-link" href="{{ url_for('auth.login')
}}">Login</a></li>
            <li class="nav-item ms-2">
              <a class="btn btn-light btn-sm" href="{{ url_for('auth.signup')
}}">Sign Up</a>
            </li>
          {% endif %}

        </ul>
      </div>
    </div>
  </nav>

  {# == FLASH MESSAGES ================================================== #}
  <div class="flash-container">
    {% with messages = get_flashed_messages(with_categories=true) %}
      {% if messages %}
        {% for category, message in messages %}
        <div class="alert alert-{{ category }} alert-dismissible fade show shadow-
sm" role="alert">
          {{ message }}
          <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
        </div>
        {% endfor %}
      {% endif %}
    {% endwith %}
  </div>

  {# == MAIN CONTENT — child templates fill this block ============== #}
  <main>
    {% block content %}{% endblock %}
  </main>
```

```
  {# == FOOTER ========================================================== #}
  <footer class="ems-footer">
    <div class="container text-center">
      <span>© 2026 EMS  |  </span>
      <a href="#">About</a>  | 
      <a href="#">Contact</a>  | 
      <a href="#">Terms</a>
    </div>
  </footer>

  {# Bootstrap 5 JS bundle (includes Popper) #}
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"><
/script>

  {# Extra scripts from child pages #}
  {% block extra_js %}{% endblock %}

  {# Auto-dismiss flash messages after 4 seconds #}
  <script>
    setTimeout(() => {
      document.querySelectorAll('.alert').forEach(a => {
        let bsAlert = bootstrap.Alert.getOrCreateInstance(a);
        bsAlert.close();
      });
    }, 4000);
  </script>

</body>
</html>
```

Part B

Write the 3 Auth HTML Templates

Each template extends base.html using Jinja2 inheritance. The {% extends %} tag pulls in the full layout. The {% block content %} tag is where the page-specific content goes.

File 1 — templates/auth/login.html

Create templates/auth/login.html in VS Code:

```
{% extends 'base.html' %}
{% block title %}Login{% endblock %}

{% block content %}
<div class="auth-wrapper">
  <div class="auth-card ems-card card p-4">

    {# Header #}
    <div class="text-center mb-4">
      <i class="bi bi-calendar-event fs-1 text-primary"></i>
      <h3 class="mt-2 fw-bold">Welcome back</h3>
      <p class="text-muted small">Sign in to your EMS account</p>
    </div>

    {# Login form #}
    <form method="POST" action="{{ url_for('auth.login') }}" novalidate>

      {# CSRF token — Flask-WTF protection #}
      <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">

      {# Email #}
      <div class="mb-3">
        <label class="form-label fw-semibold">Email address</label>
        <input type="email" name="email" class="form-control"
               placeholder="you@example.com" required autofocus
               value="{{ request.form.get('email', '') }}">
      </div>

      {# Password #}
      <div class="mb-3">
        <div class="d-flex justify-content-between">
          <label class="form-label fw-semibold">Password</label>
          <a href="{{ url_for('auth.forgot_password') }}" class="small text-
decoration-none">
            Forgot password?</a>
        </div>
        <input type="password" name="password" class="form-control"
placeholder="••••••••" required>
      </div>

      {# Role selector #}
      <div class="mb-3">
        <label class="form-label fw-semibold">I am signing in as</label>
```

```
        <select name="role_hint" class="form-select">
          <option value="attendee">Attendee</option>
          <option value="organizer">Organizer</option>
        </select>
        <div class="form-text">This helps us redirect you to the right
dashboard.</div>
      </div>

      {# Submit #}
      <div class="d-grid mb-3">
        <button type="submit" class="btn btn-primary btn-lg">
          <i class="bi bi-box-arrow-in-right me-2"></i>Login
        </button>
      </div>

    </form>

    {# Sign up link #}
    <p class="text-center text-muted small mb-0">
      Don't have an account?
      <a href="{{ url_for('auth.signup') }}" class="text-decoration-none fw-
semibold">Sign up</a>
    </p>

  </div>
</div>
{% endblock %}
```

File 2 — templates/auth/signup.html

Create templates/auth/signup.html in VS Code:

```
{% extends 'base.html' %}
{% block title %}Sign Up{% endblock %}

{% block content %}
<div class="auth-wrapper">
  <div class="auth-card ems-card card p-4">

    <div class="text-center mb-4">
      <i class="bi bi-person-plus fs-1 text-primary"></i>
      <h3 class="mt-2 fw-bold">Create an account</h3>
      <p class="text-muted small">Join EMS to register for events</p>
    </div>

    <form method="POST" action="{{ url_for('auth.signup') }}" novalidate>
      <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">

      {# Email #}
      <div class="mb-3">
        <label class="form-label fw-semibold">Email address</label>
        <input type="email" name="email" class="form-control"
               placeholder="you@example.com" required autofocus
               value="{{ request.form.get('email', '') }}">
      </div>
```

```
      {# Password #}
      <div class="mb-3">
        <label class="form-label fw-semibold">Password</label>
        <input type="password" name="password" class="form-control"
               placeholder="Min. 6 characters" required>
        <div class="form-text">At least 6 characters.</div>
      </div>

      {# Role #}
      <div class="mb-4">
        <label class="form-label fw-semibold">I am signing up as</label>
        <select name="role" class="form-select">
          <option value="attendee">Attendee — I want to attend
events</option>
          <option value="organizer">Organizer — I want to create
events</option>
        </select>
      </div>

      <div class="d-grid mb-3">
        <button type="submit" class="btn btn-primary btn-lg">
          <i class="bi bi-person-check me-2"></i>Create Account
        </button>
      </div>
    </form>

    <p class="text-center text-muted small mb-0">
      Already have an account?
      <a href="{{ url_for('auth.login') }}" class="text-decoration-none fw-
semibold">Login</a>
    </p>
  </div>
</div>
{% endblock %}
```

File 3 — templates/auth/forgot_password.html

Create templates/auth/forgot_password.html in VS Code:

```
{% extends 'base.html' %}
{% block title %}Reset Password{% endblock %}

{% block content %}
<div class="auth-wrapper">
  <div class="auth-card ems-card card p-4">

    <div class="text-center mb-4">
      <i class="bi bi-envelope-arrow-up fs-1 text-primary"></i>
      <h3 class="mt-2 fw-bold">Reset your password</h3>
      <p class="text-muted small">Enter your email and we'll send a reset
link</p>
    </div>

    {# Show success state after submission #}
    {% if sent %}
      <div class="alert alert-success text-center">
```

```
        <i class="bi bi-check-circle me-2"></i>
        Reset link sent! Check your email inbox.
      </div>
      <div class="text-center mt-3">
        <a href="{{ url_for('auth.login') }}" class="btn btn-outline-
primary">
          Back to Login
        </a>
      </div>
    {% else %}

      <form method="POST" action="{{ url_for('auth.forgot_password') }}"
novalidate>
        <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
        <div class="mb-3">
          <label class="form-label fw-semibold">Email address</label>
          <input type="email" name="email" class="form-control"
                 placeholder="you@example.com" required autofocus>
        </div>
        <div class="d-grid mb-3">
          <button type="submit" class="btn btn-primary btn-lg">
            <i class="bi bi-send me-2"></i>Send Reset Link
          </button>
        </div>
      </form>

      <p class="text-center text-muted small mb-0">
        <a href="{{ url_for('auth.login') }}" class="text-decoration-none">
          <i class="bi bi-arrow-left me-1"></i>Back to Login</a>
      </p>
    {% endif %}

  </div>
</div>
{% endblock %}
```

Part C

Write app/routes/auth.py — All 4 Auth Routes

This is the most important file today. It contains all 4 authentication routes. Create app/routes/auth.py in VS Code and paste the full code below.

Step 1 — Add your Firebase Web API key to .env

The signup and login routes use the Firebase Auth REST API, which requires your Web API key. This is different from the service account key.

- Go to Firebase Console → Project Settings (gear icon) → General tab
- Scroll down to Your apps section — if no app exists, click Add app → Web (</>)
- App nickname: ems-web → click Register app
- You will see apiKey: in the config snippet — copy that value
- Open .env in VS Code and add:

```
FIREBASE_WEB_API_KEY=AIzaSy...(your actual key here)
```

> **This key is safe to use in backend code**
> The Web API key is not secret like the service account key — it is the same key used in frontend JavaScript. But we still keep it in .env rather than hard-coding it.

Step 2 — Write app/routes/auth.py

Create app/routes/auth.py in VS Code:

```
import os, requests
from flask import (Blueprint, render_template, redirect,
                   url_for, session, flash, request)
from firebase_admin import auth as fb_auth
from app.firebase_config import db
from datetime import datetime, timezone

auth_bp = Blueprint('auth', __name__, url_prefix='/auth')

# Firebase Auth REST API base URL
FIREBASE_AUTH_URL = 'https://identitytoolkit.googleapis.com/v1/accounts:'


def get_api_key():
    return os.getenv('FIREBASE_WEB_API_KEY')


# ── SIGNUP ──────────────────────────────────────────────────────
```

```
@auth_bp.route('/signup', methods=['GET', 'POST'])
def signup():
    if request.method == 'POST':
        email    = request.form.get('email', '').strip().lower()
        password = request.form.get('password', '')
        role     = request.form.get('role', 'attendee')

        # Basic validation
        if not email or not password:
            flash('Email and password are required.', 'danger')
            return redirect(url_for('auth.signup'))

        if len(password) < 6:
            flash('Password must be at least 6 characters.', 'danger')
            return redirect(url_for('auth.signup'))

        if role not in ['attendee', 'organizer']:
            role = 'attendee'

        try:
            # Create user in Firebase Auth
            user = fb_auth.create_user(email=email, password=password)

            # Save profile to Firestore /users/{uid}
            db.collection('users').document(user.uid).set({
                'uid':              user.uid,
                'email':            email,
                'role':             role,
                'is_active':        True,
                'networking_opt_in':False,
                'created_at':       datetime.now(timezone.utc),
            })

            # Set session so user is immediately logged in
            session['uid']   = user.uid
            session['email'] = email
            session['role']  = role

            flash(f'Welcome to EMS! Your account has been created.',
'success')

            # Redirect based on role
            if role == 'organizer':
                return redirect(url_for('organizer.dashboard'))
            return redirect(url_for('attendee.my_events'))

        except fb_auth.EmailAlreadyExistsError:
            flash('An account with this email already exists. Please login.',
'warning')
            return redirect(url_for('auth.login'))
        except Exception as e:
            flash(f'Signup failed: {str(e)}', 'danger')
            return redirect(url_for('auth.signup'))

    return render_template('auth/signup.html')


# ── LOGIN ──────────────────────────────────────────────────────────
@auth_bp.route('/login', methods=['GET', 'POST'])
```

```
def login():
    if session.get('uid'):
        return redirect(url_for('public.index'))

    if request.method == 'POST':
        email    = request.form.get('email', '').strip().lower()
        password = request.form.get('password', '')

        try:
            # Call Firebase Auth REST API to verify email + password
            resp = requests.post(
                f'{FIREBASE_AUTH_URL}signInWithPassword?key={get_api_key()}',
                json={'email': email, 'password': password,
'returnSecureToken': True},
                timeout=10
            )
            data = resp.json()

            if 'error' in data:
                err = data['error']['message']
                if err in ('EMAIL_NOT_FOUND', 'INVALID_PASSWORD',
'INVALID_LOGIN_CREDENTIALS'):
                    flash('Invalid email or password.', 'danger')
                else:
                    flash(f'Login failed: {err}', 'danger')
                return redirect(url_for('auth.login'))

            # Get user profile from Firestore for role
            uid = data['localId']
            user_doc = db.collection('users').document(uid).get()
            role = 'attendee'
            if user_doc.exists:
                role = user_doc.to_dict().get('role', 'attendee')

            # Set session
            session['uid']   = uid
            session['email'] = email
            session['role']  = role

            flash(f'Welcome back!', 'success')

            if role == 'organizer':
                return redirect(url_for('organizer.dashboard'))
            if role == 'admin':
                return redirect(url_for('admin.dashboard'))
            return redirect(url_for('attendee.my_events'))

        except Exception as e:
            flash(f'Login error: {str(e)}', 'danger')
            return redirect(url_for('auth.login'))

    return render_template('auth/login.html')


# ── LOGOUT ──────────────────────────────────────────────────────────
@auth_bp.route('/logout')
def logout():
    session.clear()
    flash('You have been logged out.', 'info')
    return redirect(url_for('public.index'))
```

```python
# ── FORGOT PASSWORD ──────────────────────────────────────────────────
@auth_bp.route('/forgot-password', methods=['GET', 'POST'])
def forgot_password():
    sent = False
    if request.method == 'POST':
        email = request.form.get('email', '').strip().lower()
        try:
            requests.post(
                f'{FIREBASE_AUTH_URL}sendOobCode?key={get_api_key()}',
                json={'requestType': 'PASSWORD_RESET', 'email': email},
                timeout=10
            )
            # Always show success (do not reveal if email exists)
            sent = True
        except Exception:
            sent = True  # Still show success to prevent email enumeration
    return render_template('auth/forgot_password.html', sent=sent)
```

Part D

Write app/decorators.py — Route Guards

Decorators are functions that wrap your route functions and run before them. login_required checks if the user is logged in. role_required checks if they have the right role. You will use these on every protected route from Day 4 onwards.

Create app/decorators.py

Create app/decorators.py in VS Code:

```
from functools import wraps
from flask import session, redirect, url_for, flash


def login_required(f):
    """
    Decorator: redirect to login if user is not logged in.
    Usage:  @login_required  above any route function.
    """
    @wraps(f)
    def decorated(*args, **kwargs):
        if not session.get('uid'):
            flash('Please log in to access this page.', 'warning')
            return redirect(url_for('auth.login'))
        return f(*args, **kwargs)
    return decorated


def role_required(*roles):
    """
    Decorator: allow only users with specified role(s).
    Usage:  @role_required('organizer')  or  @role_required('organizer',
'admin')
    """
    def decorator(f):
        @wraps(f)
        def decorated(*args, **kwargs):
            if not session.get('uid'):
                flash('Please log in.', 'warning')
                return redirect(url_for('auth.login'))
            if session.get('role') not in roles:
                flash('You do not have permission to access this page.',
'danger')
                return redirect(url_for('public.index'))
            return f(*args, **kwargs)
        return decorated
    return decorator
```

How you will use these from Day 4 onwards

On every organizer route you write:

```
@login_required
@role_required('organizer')
def dashboard():
    ...
```

The decorators run top to bottom. First login_required checks the user is logged in. Then role_required checks they are an organizer. If either check fails, the user is redirected before your route code runs.

Part E

Wire Up, Rebuild Docker & Test All Flows

Step 1 — Create placeholder blueprints for missing routes

The auth routes redirect to organizer.dashboard, attendee.my_events, and public.index after login. These blueprints don't exist yet — Flask will crash without them. We create minimal placeholder files now so auth works today. We'll fill them with real content on Days 4–6.

In Git Bash:

```
touch app/routes/public.py app/routes/organizer.py app/routes/attendee.py
app/routes/admin.py
```

Open each file and paste its placeholder content:

app/routes/public.py

```
from flask import Blueprint, render_template
public_bp = Blueprint('public', __name__)

@public_bp.route('/')
def index():
    return render_template('public/index.html')
```

app/routes/organizer.py

```
from flask import Blueprint, render_template
from app.decorators import login_required, role_required
organizer_bp = Blueprint('organizer', __name__, url_prefix='/organizer')

@organizer_bp.route('/dashboard')
@login_required
@role_required('organizer')
def dashboard():
    return render_template('organizer/dashboard.html')
```

app/routes/attendee.py

```
from flask import Blueprint, render_template
from app.decorators import login_required
attendee_bp = Blueprint('attendee', __name__, url_prefix='/attendee')

@attendee_bp.route('/my-events')
@login_required
def my_events():
    return render_template('attendee/my_events.html')
```

app/routes/admin.py

```
from flask import Blueprint, render_template
from app.decorators import login_required, role_required
admin_bp = Blueprint('admin', __name__, url_prefix='/admin')

@admin_bp.route('/dashboard')
@login_required
@role_required('admin')
def dashboard():
    return render_template('admin/dashboard.html')
```

Step 2 — Create placeholder HTML templates

Each placeholder blueprint renders a template. Create these 4 minimal HTML files:

```
touch templates/public/index.html
touch templates/organizer/dashboard.html
touch templates/attendee/my_events.html
touch templates/admin/dashboard.html
```

Open each file and paste this pattern (change the title for each):

```
{% extends 'base.html' %}
{% block title %}Home{% endblock %}
{% block content %}
<div class="container py-5 text-center">
  <h2>Public Home Page</h2>
  <p class="text-muted">Full content coming on Day 5.</p>
</div>
{% endblock %}
```

> **Change the h2 text for each file**
> public/index.html → 'Public Home Page'
> organizer/dashboard.html → 'Organizer Dashboard'
> attendee/my_events.html → 'My Events'
> admin/dashboard.html → 'Admin Dashboard'

Step 3 — Update app.py to register all blueprints

Open app.py in VS Code and replace the entire file:

```
import os
from flask import Flask, jsonify
from dotenv import load_dotenv
from app.firebase_config import init_firebase
```

```
load_dotenv()

app = Flask(__name__)
app.config['SECRET_KEY'] = os.getenv('SECRET_KEY', 'dev-secret-change-this')

# Initialise Firebase
init_firebase()

# ── Register blueprints ──────────────────────────────────────────
from app.routes.test import test_bp
from app.routes.auth import auth_bp
from app.routes.public import public_bp
from app.routes.organizer import organizer_bp
from app.routes.attendee import attendee_bp
from app.routes.admin import admin_bp

app.register_blueprint(test_bp)
app.register_blueprint(auth_bp)
app.register_blueprint(public_bp)
app.register_blueprint(organizer_bp)
app.register_blueprint(attendee_bp)
app.register_blueprint(admin_bp)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```

Step 4 — Rebuild Docker and test

```
docker compose down
docker compose build
docker compose up
```

Test checklist — open browser and verify each URL:

URL	What to test	Expected result
http://localhost:5000/	Home page loads	Placeholder page with EMS navbar and footer
http://localhost:5000/auth/signup	Signup form appears	Form with email, password, role fields
Sign up with a new email	Creates account	Flash: Welcome to EMS! Redirects to My Events
http://localhost:5000/auth/logout	Logs out	Flash: You have been logged out. Redirects home
http://localhost:5000/auth/login	Login form appears	Form with email, password, role hint
Log in with the same email	Logs in	Flash: Welcome back! Redirects to dashboard
http://localhost:5000/auth/forgot-password	Reset form appears	Email input + Send Reset Link button
Submit forgot password	Shows success	Green banner: Reset link sent! Check your email

http://localhost:5000/attendee/my-events (logged out)	Route guard works	Redirects to login with flash warning

Day 3 deliverable: Sign up, log in, log out, forgot password all working. Role-based route guards redirecting correctly. Bootstrap 5 navbar shows login state.

Step 5 — Commit

```
git add .
git status  # verify secrets/ and .env NOT listed
git commit -m "Day 3: auth system, base.html, Bootstrap 5 templates, route
guards"
git push origin develop
```

End-of-Day 3 Checklist

	Item	How to verify
☐	**static/css/ems.css**	File exists with CSS variables and .auth-wrapper styles
☐	**templates/base.html**	File exists with navbar, flash messages, footer, Bootstrap 5 CDN links
☐	**auth/login.html**	Extends base.html, has email/password/role fields and forgot password link
☐	**auth/signup.html**	Extends base.html, has email/password/role selector
☐	**auth/forgot_password.html**	Extends base.html, shows success state after submit
☐	**FIREBASE_WEB_API_KEY in .env**	The key is set to your real Web API key from Firebase Console
☐	**app/routes/auth.py**	File exists with all 4 routes: signup, login, logout, forgot_password
☐	**app/decorators.py**	File exists with login_required and role_required functions
☐	**4 placeholder blueprints**	public.py, organizer.py, attendee.py, admin.py all exist in app/routes/
☐	**4 placeholder templates**	public/index.html, organizer/dashboard.html, attendee/my_events.html, admin/dashboard.html
☐	**app.py updated**	All 6 blueprints registered
☐	**Docker builds cleanly**	docker compose build completes with no errors
☐	**Signup works**	New user created, redirected to My Events page
☐	**Login works**	Existing user logs in, redirected to correct dashboard
☐	**Logout works**	Session cleared, redirected to home with flash message
☐	**Forgot password works**	Success message shown, reset email received in inbox
☐	**Route guard works**	Visiting /attendee/my-events while logged out redirects to login
☐	**Firebase Console**	New user appears in Firebase Console → Authentication → Users
☐	**Committed & pushed**	GitHub develop branch shows Day 3 commit

What's Next — Day 4 Preview

Tomorrow you build the full authentication flow completion and start on venue management — the first real database write operation from the organizer's perspective.

- Password reset flow completion and email verification
- Venue add, edit, delete routes with Firestore writes
- Venue list page for organizers
- Form validation with user-friendly error messages
- First real use of @login_required and @role_required('organizer') decorators on real routes

Tag and backup reminder

After completing and committing Day 3, run:

```
git tag v0.0.3-day3 -m 'Day 3 complete — auth system working'
git push origin v0.0.3-day3
cp -r /d/projects/event-management-system /d/projects/event-management-system-day3-backup
```

Day 4 — Setup Guide

Venue Management

Firestore CRUD | Organizer Routes | 4 Parts | ~5 hours

Part A	Write app/routes/venues.py — all 4 venue routes (~90 min)
Part B	Write templates/organizer/venues/ — list and form pages (~60 min)
Part C	Write app/utils/validators.py — reusable form validation (~30 min)
Part D	Wire up, rebuild Docker, test all venue flows in browser (~30 min)
End goal	Organizer can add, edit, delete venues. All changes persist in Firestore.

Overview — What we are building today

Venue management is the first complete CRUD (Create, Read, Update, Delete) feature of your EMS. It is also the first time you will write real Firestore operations from an organizer's perspective with proper route guards.

Today you will write code that:

- Lists all venues belonging to the logged-in organizer from Firestore
- Creates a new venue document in Firestore with server timestamp
- Edits an existing venue — with ownership check (organizer can only edit their own)
- Deletes a venue — with ownership check and confirmation guard
- Validates all form inputs server-side with clear error messages

New files created today

```
event-management-system/
├── app/
│   ├── routes/
│   │   └── venues.py          ← NEW — all 4 venue routes
│   └── utils/                 ← NEW folder
│       ├── __init__.py        ← NEW
│       └── validators.py      ← NEW — reusable form validation
├── templates/organizer/
│   └── venues/                ← NEW folder
│       ├── list.html          ← NEW — venue list page
│       └── form.html          ← NEW — shared add/edit form
└── app.py                     (updated — register venues blueprint)
```

> **Wireframe reference**
> Today's pages come from wireframe page 7 (Create/Edit Event page shows the venue dropdown and Add New Venue option) and wireframe page 8 (Manage Ticket Types references venue). The venue management pages themselves are the organizer back-office screens.

Part A

Write app/routes/venues.py — All 4 Venue Routes

Create a new file app/routes/venues.py in VS Code. This file contains all venue-related routes. Read the comments carefully — they explain every Firestore operation.

First — create the utils folder

In Git Bash:

```
mkdir -p app/utils
touch app/utils/__init__.py app/utils/validators.py
touch app/routes/venues.py
mkdir -p templates/organizer/venues
touch templates/organizer/venues/list.html
templates/organizer/venues/form.html
```

Now write app/routes/venues.py

```
from flask import (Blueprint, render_template, redirect,
                        url_for, session, flash, request)
from app.firebase_config import db
from app.decorators import login_required, role_required
from app.utils.validators import validate_venue
from datetime import datetime, timezone
from google.cloud.firestore import SERVER_TIMESTAMP

venues_bp = Blueprint('venues', __name__, url_prefix='/organizer/venues')


# ── Helper: get venue and verify ownership ──────────────────────────
def get_venue_or_404(venue_id):
    doc = db.collection('venues').document(venue_id).get()
    if not doc.exists:
        return None, None
    data = doc.to_dict()
    # Ownership check: only the organizer who created it can modify it
    if data.get('organizer_uid') != session.get('uid'):
        return None, 'forbidden'
    return doc, data


# ── LIST all venues for this organizer ──────────────────────────────
@venues_bp.route('/')
@login_required
@role_required('organizer')
def list_venues():
    # Query only venues owned by the logged-in organizer
    docs = (
        db.collection('venues')
        .where('organizer_uid', '==', session['uid'])
        .order_by('name')
```

```
        .stream()
    )
    venues = [{**d.to_dict(), 'id': d.id} for d in docs]
    return render_template('organizer/venues/list.html', venues=venues)


# ── CREATE new venue (GET = show form, POST = save) ──────────────────
@venues_bp.route('/create', methods=['GET', 'POST'])
@login_required
@role_required('organizer')
def create_venue():
    if request.method == 'POST':
        form_data = request.form.to_dict()
        errors = validate_venue(form_data)

        if errors:
            for err in errors:
                flash(err, 'danger')
            return render_template('organizer/venues/form.html',
                                   form_data=form_data, action='create')

        # Save to Firestore
        db.collection('venues').add({
            'name':          form_data['name'].strip(),
            'address':       form_data['address'].strip(),
            'city':          form_data['city'].strip(),
            'capacity':      int(form_data['capacity']),
            'contact_name':  form_data.get('contact_name', '').strip(),
            'contact_phone': form_data.get('contact_phone', '').strip(),
            'organizer_uid': session['uid'],
            'created_at':    SERVER_TIMESTAMP,
        })

        flash(f"Venue '{form_data['name']}' created successfully!",
'success')
        return redirect(url_for('venues.list_venues'))

    return render_template('organizer/venues/form.html',
                           form_data={}, action='create')


# ── EDIT existing venue ──────────────────────────────────────────────
@venues_bp.route('/<venue_id>/edit', methods=['GET', 'POST'])
@login_required
@role_required('organizer')
def edit_venue(venue_id):
    doc, data = get_venue_or_404(venue_id)

    if doc is None:
        flash('Venue not found or access denied.', 'danger')
        return redirect(url_for('venues.list_venues'))

    if request.method == 'POST':
        form_data = request.form.to_dict()
        errors = validate_venue(form_data)

        if errors:
            for err in errors:
                flash(err, 'danger')
            return render_template('organizer/venues/form.html',
```

```
                                form data=form data, action='edit',
venue_id=venue_id)

        # Update only the editable fields (never overwrite organizer_uid or
created_at)
        db.collection('venues').document(venue_id).update({
            'name':          form_data['name'].strip(),
            'address':       form_data['address'].strip(),
            'city':          form_data['city'].strip(),
            'capacity':      int(form_data['capacity']),
            'contact_name':  form_data.get('contact_name', '').strip(),
            'contact_phone': form_data.get('contact_phone', '').strip(),
            'updated_at':    SERVER_TIMESTAMP,
        })

        flash(f"Venue '{form_data['name']}' updated successfully!",
'success')
        return redirect(url_for('venues.list_venues'))

    # GET: pre-fill form with existing data
    return render_template('organizer/venues/form.html',
                           form_data=data, action='edit', venue_id=venue_id)


# ── DELETE venue ──────────────────────────────────────────────────────
@venues_bp.route('/<venue_id>/delete', methods=['POST'])
@login_required
@role_required('organizer')
def delete_venue(venue_id):
    doc, data = get_venue_or_404(venue_id)

    if doc is None:
        flash('Venue not found or access denied.', 'danger')
        return redirect(url_for('venues.list_venues'))

    venue_name = data.get('name', 'this venue')
    db.collection('venues').document(venue_id).delete()
    flash(f"Venue '{venue_name}' deleted.", 'info')
    return redirect(url_for('venues.list_venues'))
```

Why DELETE uses POST not GET

Browsers can accidentally trigger GET requests (prefetch, history, bots). Using POST for delete means only an intentional form submission can delete data. The delete button in the HTML will be inside a small form with method=POST.

Part B

Write the 2 Venue HTML Templates

File 1 — templates/organizer/venues/list.html

This page shows a table of all venues owned by the organizer — matching wireframe page 8 style. Open the file in VS Code and paste this:

```
{% extends 'base.html' %}
{% block title %}My Venues{% endblock %}

{% block content %}
<div class="container py-4">

  {# Page header #}
  <div class="d-flex justify-content-between align-items-center mb-4">
    <div>
      <h2 class="fw-bold mb-1">My Venues</h2>
      <p class="text-muted mb-0">Manage the venues available for your
events.</p>
    </div>
    <a href="{{ url_for('venues.create_venue') }}" class="btn btn-primary">
      <i class="bi bi-plus-lg me-2"></i>Add New Venue
    </a>
  </div>

  {# Empty state #}
  {% if not venues %}
  <div class="ems-card card text-center py-5">
    <i class="bi bi-building fs-1 text-muted mb-3"></i>
    <h5 class="text-muted">No venues yet</h5>
    <p class="text-muted small">Add your first venue to get started.</p>
    <a href="{{ url_for('venues.create_venue') }}" class="btn btn-primary
btn-sm">
      Add First Venue</a>
  </div>

  {% else %}
  <div class="ems-card card">
    <div class="table-responsive">
      <table class="table table-hover align-middle mb-0">
        <thead class="table-dark">
          <tr>
            <th>Name</th>
            <th>City</th>
            <th>Capacity</th>
            <th>Contact</th>
            <th class="text-end">Actions</th>
          </tr>
        </thead>
        <tbody>
          {% for venue in venues %}
          <tr>
            <td>
              <strong>{{ venue.name }}</strong>
```

```
              <div class="text-muted small">{{ venue.address }}</div>
            </td>
            <td>{{ venue.city }}</td>
            <td><span class="badge bg-secondary">{{ venue.capacity }}
seats</span></td>
            <td>
              {% if venue.contact_name %}
                {{ venue.contact_name }}
                {% if venue.contact_phone %}<div class="text-muted small">{{
venue.contact_phone }}</div>{% endif %}
              {% else %}
                <span class="text-muted">—</span>
              {% endif %}
            </td>
            <td class="text-end">
              {# Edit button #}
              <a href="{{ url_for('venues.edit_venue', venue_id=venue.id) }}"
                 class="btn btn-sm btn-outline-primary me-1">
                <i class="bi bi-pencil me-1"></i>Edit
              </a>
              {# Delete form — POST only, with confirmation dialog #}
              <form method="POST"
                    action="{{ url_for('venues.delete_venue',
venue_id=venue.id) }}"
                    class="d-inline"
                    onsubmit="return confirm('Delete venue: {{ venue.name
}}?')">
                <input type="hidden" name="csrf_token" value="{{ csrf_token()
}}">
                <button type="submit" class="btn btn-sm btn-outline-danger">
                  <i class="bi bi-trash me-1"></i>Delete
                </button>
              </form>
            </td>
          </tr>
          {% endfor %}
        </tbody>
      </table>
    </div>
  </div>
  {% endif %}

</div>
{% endblock %}
```

File 2 — templates/organizer/venues/form.html

This single form handles both Add and Edit — the `action` variable passed from the route controls which mode it is in. Open the file and paste:

```
{% extends 'base.html' %}
{% block title %}{{ 'Edit' if action == 'edit' else 'Add' }} Venue{% endblock
%}

{% block content %}
<div class="container py-4">
  <div class="row justify-content-center">
```

```
    <div class="col-lg-7">

      {# Breadcrumb #}
      <nav aria-label="breadcrumb" class="mb-3">
        <ol class="breadcrumb">
          <li class="breadcrumb-item">
            <a href="{{ url_for('venues.list_venues') }}">My Venues</a></li>
          <li class="breadcrumb-item active">
            {{ 'Edit Venue' if action == 'edit' else 'Add New Venue' }}</li>
        </ol>
      </nav>

      <div class="ems-card card p-4">
        <h4 class="fw-bold mb-4">
          <i class="bi bi-building me-2 text-primary"></i>
          {{ 'Edit Venue' if action == 'edit' else 'Add New Venue' }}
        </h4>

        {% if action == 'edit' %}
          <form method="POST" action="{{ url_for('venues.edit_venue',
venue_id=venue_id) }}" novalidate>
        {% else %}
          <form method="POST" action="{{ url_for('venues.create_venue') }}"
novalidate>
        {% endif %}

          <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">

          {# Venue Name #}
          <div class="mb-3">
            <label class="form-label fw-semibold">Venue Name <span
class="text-danger">*</span></label>
            <input type="text" name="name" class="form-control"
                   placeholder="e.g. City Hall, Udupi"
                   value="{{ form_data.get('name', '') }}" required>
          </div>

          {# Address #}
          <div class="mb-3">
            <label class="form-label fw-semibold">Address <span class="text-
danger">*</span></label>
            <input type="text" name="address" class="form-control"
                   placeholder="Street address"
                   value="{{ form_data.get('address', '') }}" required>
          </div>

          {# City and Capacity side by side #}
          <div class="row">
            <div class="col-md-7 mb-3">
              <label class="form-label fw-semibold">City <span class="text-
danger">*</span></label>
              <input type="text" name="city" class="form-control"
                     placeholder="e.g. Udupi"
                     value="{{ form_data.get('city', '') }}" required>
            </div>
            <div class="col-md-5 mb-3">
              <label class="form-label fw-semibold">Capacity <span
class="text-danger">*</span></label>
              <input type="number" name="capacity" class="form-control"
                     placeholder="e.g. 500" min="1"
```

```
                    value="{{ form data.get('capacity', '') }}" required>
            </div>
          </div>

          {# Optional contact fields #}
          <hr class="my-3">
          <p class="text-muted small mb-3">Contact information (optional)</p>
          <div class="row">
            <div class="col-md-6 mb-3">
              <label class="form-label fw-semibold">Contact Name</label>
              <input type="text" name="contact_name" class="form-control"
                     placeholder="Contact person"
                     value="{{ form_data.get('contact_name', '') }}">
            </div>
            <div class="col-md-6 mb-3">
              <label class="form-label fw-semibold">Contact Phone</label>
              <input type="tel" name="contact_phone" class="form-control"
                     placeholder="+91 XXXXX XXXXX"
                     value="{{ form_data.get('contact_phone', '') }}">
            </div>
          </div>

          {# Buttons #}
          <div class="d-flex gap-2 mt-2">
            <button type="submit" class="btn btn-primary">
              <i class="bi bi-check-lg me-2"></i>
              {{ 'Save Changes' if action == 'edit' else 'Create Venue' }}
            </button>
            <a href="{{ url_for('venues.list_venues') }}" class="btn btn-
outline-secondary">
              Cancel
            </a>
          </div>

        </form>
      </div>
    </div>
  </div>
</div>
{% endblock %}
```

Part C

Write app/utils/validators.py — Form Validation

Validation logic lives in a separate utils file so it can be reused across all routes. When you build event creation (Day 6), ticket types (Day 8), and promo codes (Week 4) you will add more validators to this same file.

Write app/utils/validators.py

Open app/utils/validators.py in VS Code and paste this:

```
# ── Venue validator ──────────────────────────────────────────────
def validate_venue(data):
    """
    Validate venue form data.
    Returns a list of error strings.
    Empty list means validation passed.
    """
    errors = []

    name = data.get('name', '').strip()
    address = data.get('address', '').strip()
    city = data.get('city', '').strip()
    capacity = data.get('capacity', '').strip()

    if not name:
        errors.append('Venue name is required.')
    elif len(name) < 3:
        errors.append('Venue name must be at least 3 characters.')
    elif len(name) > 100:
        errors.append('Venue name must be under 100 characters.')

    if not address:
        errors.append('Address is required.')

    if not city:
        errors.append('City is required.')

    if not capacity:
        errors.append('Capacity is required.')
    else:
        try:
            cap = int(capacity)
            if cap < 1:
                errors.append('Capacity must be at least 1.')
            elif cap > 1000000:
                errors.append('Capacity value seems unrealistically large.')
        except ValueError:
            errors.append('Capacity must be a whole number.')

    return errors
```

```
# ── Event validator (stub — will be filled on Day 6) ─────────────────
def validate_event(data):
    # TODO: implement on Day 6
    return []


# ── Ticket type validator (stub — will be filled on Day 8) ───────────
def validate_ticket_type(data):
    # TODO: implement on Day 8
    return []
```

Part D

Wire Up, Rebuild Docker & Test All Venue Flows

Step 1 — Register the venues blueprint in app.py

Open app.py in VS Code. Find the blueprint registration section and add two lines:

Add this import with the others:

```
from app.routes.venues import venues_bp
```

Add this registration line with the others:

```
app.register_blueprint(venues_bp)
```

> **Order matters for blueprints**
> Register venues_bp after auth_bp and public_bp. Blueprint registration order affects Flask's URL routing — keeping them in logical order (auth → public → organizer routes) avoids any future conflicts.

Step 2 — Add venues link to organizer navbar in base.html

Open templates/base.html in VS Code. Find the organizer navbar section (the {% if session.get('role') == 'organizer' %} block) and add a Venues link after the Dashboard link:

```
<li class="nav-item">
  <a class="nav-link" href="{{ url_for('venues.list_venues') }}">
    <i class="bi bi-building me-1"></i>Venues
  </a>
</li>
```

Step 3 — Create a Firestore index

The venues list route queries Firestore with .where() and .order_by() together. Firestore requires a composite index for this combination. When you first run the query, Firestore will print an error with a direct link to create the index automatically.

- Run docker compose up and visit the venues page
- If you see a Firestore index error in Git Bash output, copy the URL from the error message
- Open that URL in your browser — it takes you straight to Firebase Console to create the index
- Click Create index — it takes about 1-2 minutes to build
- Refresh the venues page — it will work once the index is ready

> **You will only do this once per query combination**
> Once the index is created in Firebase Console it persists forever. You never need to create it again. This is a one-time setup step specific to Firestore's NoSQL architecture.

Step 4 — Rebuild and test

```
docker compose down
docker compose build
docker compose up
```

Test every flow in the browser:

Action	What to test	Expected result
Log in as organizer	Use account created on Day 3	Redirects to organizer dashboard
Click Venues in navbar	Visits /organizer/venues/	Empty state page with Add New Venue button
Click Add New Venue	Form loads	Form with Name, Address, City, Capacity fields
Submit form with empty Name	Validation check	Flash: Venue name is required. Form re-renders
Submit form with Capacity = abc	Validation check	Flash: Capacity must be a whole number.
Fill form correctly and submit	Creates venue	Flash: Venue created! Venue appears in list
Click Edit on the venue	Edit form loads	Form pre-filled with existing venue data
Change the city and save	Updates venue	Flash: Venue updated! New city shows in list
Click Delete	Confirmation dialog	Browser shows: Delete venue: [name]?
Confirm delete	Deletes venue	Flash: Venue deleted. List is empty again
Visit /organizer/venues/ logged out	Route guard	Redirects to login with warning flash
Log in as attendee, visit /organizer/venues/	Role guard	Redirects to home with permission error flash

> **Day 4 deliverable: Full venue CRUD working. Organizer can add, edit, delete venues. All data persists in Firestore. Route guards blocking attendees and logged-out users.**

Step 5 — Verify in Firebase Console

- Go to Firebase Console → Firestore Database
- Click the venues collection (created automatically when you added your first venue)

- You should see your venue document with all fields including `organizer_uid` and `created_at`
- This confirms Firestore writes are working correctly

Step 6 — Commit

```
git add .
git status   # confirm secrets/ and .env not listed
git commit -m "Day 4: venue CRUD, Firestore writes, form validation, route
guards"
git push origin develop
```

End-of-Day 4 Checklist

	Item	How to verify
☐	**app/utils/__init__.py**	Empty file exists at app/utils/__init__.py
☐	**app/utils/validators.py**	Contains validate_venue() returning list of errors
☐	**app/routes/venues.py**	Contains list_venues, create_venue, edit_venue, delete_venue routes
☐	**templates/organizer/venues/list.html**	Shows table with Edit/Delete buttons and empty state
☐	**templates/organizer/venues/form.html**	Single form handles both add and edit via action variable
☐	**venues_bp registered in app.py**	app.py imports and registers venues_bp
☐	**Venues link in navbar**	Navbar shows Venues link for organizer role
☐	**Docker builds cleanly**	docker compose build completes without errors
☐	**Venue list loads**	Visiting /organizer/venues/ shows the list page
☐	**Create venue works**	New venue saved to Firestore, appears in list
☐	**Validation works**	Empty name shows flash error, form re-renders with data
☐	**Edit venue works**	Changes saved to Firestore, list shows updated data
☐	**Delete venue works**	Confirmation dialog appears, venue removed from Firestore
☐	**Firestore Console**	venues collection visible in Firebase Console with correct fields
☐	**Route guard — logged out**	Visiting /organizer/venues/ without login → redirects to login
☐	**Role guard — attendee**	Attendee visiting /organizer/venues/ → redirects with error flash
☐	**Committed & pushed**	GitHub develop branch shows Day 4 commit

What's Next — Day 5 Preview

Tomorrow you build the public event listing page and the event detail page — the two pages attendees see first (wireframe pages 1 and 2). This is the first day you write pages that display real event data from Firestore to the public.

- Public home page (/) with event cards grid — wireframe page 1
- Event detail page with ticket type cards — wireframe page 2
- Firestore query: published events, ordered by start date
- Replace the placeholder public/index.html with the real event listing
- Add venue name lookup (cross-reference venues collection)

Tag and backup reminder

After completing Day 4:

```
git tag v0.0.4-day4 -m 'Day 4 complete — venue CRUD working'
git push origin v0.0.4-day4
cp -r /d/projects/event-management-system /d/projects/event-management-system-day4-backup
```

Day 5 — Setup Guide

Public Event Listing & Event Detail

Wireframe Pages 1 & 2 | 4 Parts | ~5 hours

Part A	Write app/routes/public.py — event listing & detail routes (~60 min)
Part B	Write templates/public/index.html — event cards grid (~60 min)
Part C	Write templates/public/event_detail.html — ticket selection page (~60 min)
Part D	Wire up, add CSS card styles, rebuild Docker, test in browser (~30 min)
End goal	Public can browse events and view ticket types. Logged-in users see Register button.

Overview — What we are building today

Today you replace the two placeholder public pages with real, data-driven pages that read from Firestore. These are the first pages an attendee sees — wireframe pages 1 and 2.

Page	URL	What it shows
Public Home (pg.1)	/	Grid of event cards — name, date, venue, Register button
Event Detail (pg.2)	/events/<event_id>	Full event info, ticket type cards, promo code input, Proceed button

> **Wireframe fidelity today**
> Page 1 wireframe shows 4 event cards in a 2x2 grid with search and date filter. Today we build the cards grid and search. The date filter dropdown comes on Day 6 alongside the full agenda. Page 2 wireframe shows ticket type cards with Select button, promo code input, and Proceed button — we build all of these today.

New files today

```
event-management-system/
├── app/routes/
│   └── public.py              ← REPLACE placeholder with real routes
├── templates/public/
│   ├── index.html             ← REPLACE placeholder with event cards
grid
│   └── event_detail.html      ← NEW — full event + ticket selection page
└── static/css/ems.css         ← ADD event card styles
```

Part A

Write app/routes/public.py — Event Listing & Detail Routes

Open app/routes/public.py in VS Code and replace the entire placeholder file with this:

```
from flask import (Blueprint, render_template, request,
                        abort, flash, redirect, url_for, session)
from app.firebase_config import db

public_bp = Blueprint('public', __name__)


# ── PUBLIC HOME — event listing ──────────────────────────────────────
@public_bp.route('/')
def index():
    search = request.args.get('q', '').strip().lower()

    # Fetch all published events ordered by start date
    docs = (
        db.collection('events')
        .where('status', '==', 'published')
        .order_by('start_datetime')
        .stream()
    )

    events = []
    for doc in docs:
        data = {**doc.to_dict(), 'id': doc.id}

        # Get venue name for display on the card
        venue_name = 'Venue TBD'
        if data.get('venue_id'):
            v = db.collection('venues').document(data['venue_id']).get()
            if v.exists:
                venue_name = v.to_dict().get('name', 'Unknown Venue')
        data['venue_name'] = venue_name

        # Client-side keyword search filter
        if search:
            searchable = f"{data.get('name','')} {venue_name}".lower()
            if search not in searchable:
                continue

        events.append(data)

    return render_template('public/index.html',
                           events=events, search=search)


# ── EVENT DETAIL PAGE ────────────────────────────────────────────────
@public_bp.route('/events/<event_id>')
def event_detail(event_id):
    # Fetch the event document
    doc = db.collection('events').document(event_id).get()
```

```
    if not doc.exists:
        abort(404)

    event = {**doc.to_dict(), 'id': doc.id}

    # Only show published events to the public
    if event.get('status') != 'published':
        abort(404)

    # Fetch venue details
    venue = None
    if event.get('venue_id'):
        v = db.collection('venues').document(event['venue_id']).get()
        if v.exists:
            venue = v.to_dict()

    # Fetch ticket types from subcollection
    ticket_docs = (
        db.collection('events')
        .document(event_id)
        .collection('ticket_types')
        .where('is active', '==', True)
        .stream()
    )
    ticket_types = [{**t.to_dict(), 'id': t.id} for t in ticket_docs]

    # Calculate availability for each ticket type
    for tt in ticket_types:
        available = tt.get('quantity_total', 0) - tt.get('quantity_sold', 0)
        tt['available'] = max(0, available)
        tt['sold_out']  = available <= 0

    return render_template('public/event_detail.html',
                           event=event,
                           venue=venue,
                           ticket_types=ticket_types)
```

Why we look up venue inside the loop

Firestore does not support SQL-style joins. For each event we make a separate read to get the venue name. For MVP this is fine (few events). From Sprint 6 onwards we will denormalise venue_name directly onto the event document to avoid this extra read.

Part B

Write templates/public/index.html — Event Cards Grid

Open templates/public/index.html and replace the entire placeholder file with this. This is wireframe page 1.

```
{% extends 'base.html' %}
{% block title %}Events{% endblock %}

{% block content %}

{# ── Hero banner ──────────────────────────────────────────────────── #}
<div class="bg-primary text-white py-5">
  <div class="container text-center">
    <h1 class="fw-bold display-5">Upcoming Events</h1>
    <p class="lead mb-4">Discover and register for B2B events near you.</p>

    {# Search bar — wireframe page 1 #}
    <form method="GET" action="{{ url for('public.index') }}" class="row
justify-content-center g-2">
      <div class="col-md-6">
        <input type="text" name="q" class="form-control form-control-lg"
               placeholder="Search events by name or venue..."
               value="{{ search }}">
      </div>
      <div class="col-auto">
        <button type="submit" class="btn btn-light btn-lg">
          <i class="bi bi-search me-1"></i>Search
        </button>
      </div>
      {% if search %}
      <div class="col-auto">
        <a href="{{ url_for('public.index') }}" class="btn btn-outline-light
btn-lg">Clear</a>
      </div>
      {% endif %}
    </form>
  </div>
</div>

{# ── Events grid ──────────────────────────────────────────────────── #}
<div class="container py-5">

  {# Search result label #}
  {% if search %}
  <p class="text-muted mb-4">
    Showing results for <strong>"{{ search }}"</strong>
     ({{ events|length }} found)
  </p>
  {% endif %}

  {# Empty state #}
  {% if not events %}
  <div class="text-center py-5">
    <i class="bi bi-calendar-x fs-1 text-muted mb-3 d-block"></i>
```

```
    <h4 class="text-muted">No events found</h4>
    {% if search %}
    <p class="text-muted">Try a different search term.</p>
    {% else %}
    <p class="text-muted">Check back soon for upcoming events.</p>
    {% endif %}
  </div>

  {% else %}
  <div class="row row-cols-1 row-cols-md-2 row-cols-lg-3 g-4">
    {% for event in events %}
    <div class="col">
      <div class="card ems-card h-100">

        {# Cover image or placeholder #}
        {% if event.cover_image_url %}
        <img src="{{ event.cover_image_url }}" class="card-img-top ems-event-
img" alt="{{ event.name }}">
        {% else %}
        <div class="ems-event-img-placeholder bg-primary bg-gradient d-flex
align-items-center justify-content-center">
          <i class="bi bi-calendar-event text-white" style="font-
size:3rem"></i>
        </div>
        {% endif %}

        <div class="card-body d-flex flex-column">
          <h5 class="card-title fw-bold">{{ event.name }}</h5>

          {# Date and venue row #}
          <p class="card-text text-muted small mb-1">
            <i class="bi bi-calendar3 me-1"></i>
            {% if event.start_datetime %}
              {{ event.start_datetime.strftime('%b %d, %Y') }}
            {% else %}Date TBD{% endif %}
          </p>
          <p class="card-text text-muted small mb-3">
            <i class="bi bi-geo-alt me-1"></i>{{ event.venue_name }}
          </p>

          {# Description snippet #}
          <p class="card-text text-muted small flex-grow-1">
            {{ event.description[:120] }}{% if event.description|length > 120
%}...{% endif %}
          </p>

          {# Action buttons — wireframe shows View Details and Register #}
          <div class="d-flex gap-2 mt-3">
            <a href="{{ url_for('public.event_detail', event_id=event.id) }}"
               class="btn btn-outline-primary btn-sm flex-fill">
              View Details
            </a>
            {% if session.get('uid') %}
            <a href="{{ url_for('public.event_detail', event_id=event.id) }}"
               class="btn btn-primary btn-sm flex-fill">
              Register
            </a>
            {% else %}
            <a href="{{ url_for('auth.login') }}"
               class="btn btn-primary btn-sm flex-fill">
              Login to Register
```

```
            </a>
            {% endif %}
          </div>
        </div>
      </div>
    </div>
    {% endfor %}
  </div>
  {% endif %}
</div>
{% endblock %}
```

Part C

Write templates/public/event_detail.html — Event Detail & Ticket Selection

Create templates/public/event_detail.html in VS Code. This is wireframe page 2 — full event info with ticket type cards, promo code input, and the Proceed to Registration button.

```
{% extends 'base.html' %}
{% block title %}{{ event.name }}{% endblock %}

{% block content %}
<div class="container py-4">

  {# Back link #}
  <a href="{{ url_for('public.index') }}" class="text-decoration-none text-
muted mb-3 d-inline-block">
    <i class="bi bi-arrow-left me-1"></i>Back to Events
  </a>

  <div class="row g-4">

    {# ── LEFT COLUMN — event info ──────────────────────── #}
    <div class="col-lg-7">

      {# Cover image #}
      {% if event.cover_image_url %}
      <img src="{{ event.cover_image_url }}" class="img-fluid rounded-3 mb-4
w-100" style="max-height:320px;object-fit:cover" alt="{{ event.name }}">
      {% endif %}

      {# Event title and meta #}
      <h2 class="fw-bold">{{ event.name }}</h2>
      <div class="d-flex flex-wrap gap-3 text-muted mb-4">
        <span><i class="bi bi-calendar3 me-1"></i>
          {% if event.start_datetime %}
            {{ event.start_datetime.strftime('%B %d, %Y  %I:%M %p') }}
          {% endif %}
        </span>
        {% if venue %}
        <span><i class="bi bi-geo-alt me-1"></i>{{ venue.name }}, {{
venue.city }}</span>
        {% endif %}
        {% if event.event_type %}
        <span class="badge bg-info text-dark">{{ event.event_type |
capitalize }}</span>
        {% endif %}
      </div>

      {# Description box — wireframe shows this in a bordered container #}
      <div class="ems-card card p-3 mb-4">
        <h6 class="fw-bold text-muted text-uppercase small mb-2">About this
event</h6>
        <p class="mb-0" style="white-space:pre-line">{{ event.description
}}</p>
```

```
      </div>

    </div>

    {# ── RIGHT COLUMN — ticket selection ─────────────── #}
    <div class="col-lg-5">
      <div class="ems-card card p-4 sticky-top" style="top:80px">
        <h5 class="fw-bold mb-3">
          <i class="bi bi-ticket-perforated me-2 text-primary"></i>Ticket
Types
        </h5>

        {# No tickets available #}
        {% if not ticket types %}
        <p class="text-muted">No tickets available yet.</p>
        {% else %}

        {# Ticket type cards — wireframe page 2 #}
        {% for tt in ticket_types %}
        <div class="ems-ticket-card mb-3 p-3 border rounded-3 {{ 'opacity-50'
if tt.sold_out }}">
          <div class="d-flex justify-content-between align-items-start">
            <div>
              <h6 class="fw-bold mb-1">{{ tt.name }}</h6>
              {% if tt.description %}
              <p class="text-muted small mb-1">{{ tt.description }}</p>
              {% endif %}
              <p class="text-muted small mb-0">
                {% if tt.sold out %}
                  <span class="text-danger fw-semibold">Sold Out</span>
                {% else %}
                  {{ tt.available }} tickets left
                   |  Max {{ tt.max_per_order }} per person
                {% endif %}
              </p>
            </div>
            <div class="text-end ms-3">
              <div class="fs-5 fw-bold text-primary">
                {% if tt.price == 0 %}Free{% else %}₹{{ tt.price }}{% endif
%}
              </div>
            </div>
          </div>

          {# Select button — only shown if not sold out and user is logged in
#}
          {% if not tt.sold_out %}
            {% if session.get('uid') %}
            <a href="{{ url_for('registration.register', event_id=event.id,
ticket_type_id=tt.id) }}"
               class="btn btn-outline-primary btn-sm mt-2 w-100">
              <i class="bi bi-check-circle me-1"></i>Select & Register
            </a>
            {% else %}
            <a href="{{ url_for('auth.login') }}"
               class="btn btn-outline-secondary btn-sm mt-2 w-100">
              Login to Register
            </a>
            {% endif %}
          {% endif %}
        </div>
```

```
        {% endfor %}

        {# Promo code input — wireframe page 2 #}
        <hr class="my-3">
        <label class="form-label small fw-semibold">Have a promo
code?</label>
        <div class="input-group input-group-sm">
          <input type="text" id="promoInput" class="form-control text-
uppercase"
                 placeholder="Enter code">
          <button class="btn btn-outline-secondary" type="button"
                  onclick="applyPromo()">Apply</button>
        </div>
        <div id="promoMsg" class="form-text mt-1"></div>

        {% endif %}
      </div>
    </div>
  </div>
</div>

{% block extra_js %}
<script>
  {# Promo code UI — validation wired to backend on Day 9 #}
  function applyPromo() {
    const code = document.getElementById('promoInput').value.trim();
    const msg  = document.getElementById('promoMsg');
    if (!code) { msg.textContent = 'Please enter a promo code.'; return; }
    msg.textContent = 'Promo code validation coming on Day 9.';
    msg.className = 'form-text text-info mt-1';
  }
</script>
{% endblock %}
{% endblock %}
```

Part D

Add Card Styles, Create Test Event & Test in Browser

Step 1 — Add card styles to static/css/ems.css

Open static/css/ems.css and add these lines at the bottom:

```
/* ── Event cards ──────────────────────────────────────────────── */
.ems-event-img { height: 180px; object-fit: cover; width: 100%; }
.ems-event-img-placeholder { height: 180px; width: 100%; }

/* ── Ticket type cards ────────────────────────────────────────── */
.ems-ticket-card { transition: border-color 0.2s; cursor: pointer; }
.ems-ticket-card:hover { border-color: var(--ems-primary) !important; }

/* ── Responsive tweaks ────────────────────────────────────────── */
@media (max-width: 768px) {
  .sticky-top { position: relative !important; top: 0 !important; }
}
```

Step 2 — Create a Firestore index for public event listing

The public listing route queries Firestore with .where('status','==','published') and .order_by('start_datetime'). This needs a composite index — same as the venues index on Day 4.

- Run docker compose up and visit http://localhost:5000
- If you see an index error in the Docker logs, copy the URL from the error message
- Open that URL in your browser → click Create index → wait 1-2 minutes
- Refresh the home page — it will work once the index shows Enabled

> **Manual index creation if needed**
> Collection ID: events
> Field 1: status — Ascending
> Field 2: start_datetime — Ascending
> Query scopes: Collection

Step 3 — Create a test event in Firestore to see the listing

You cannot see the event listing until at least one published event exists. We will create one manually in Firebase Console so you can test today without waiting for Day 6 (event creation form).

- Go to Firebase Console → Firestore Database → Data tab
- Click + Start collection → Collection ID: events → click Next
- Document ID: click Auto-ID → click Save

- Now add these fields to the document one by one:

Field name	Type	Value to enter
name	string	TechConf 2026
description	string	Annual technology conference featuring keynotes, workshops, and networking.
status	string	published
event_type	string	physical
start_datetime	timestamp	Pick any future date from the date picker
end_datetime	timestamp	Pick a date after start
venue_id	string	Paste the ID of a venue you created on Day 4 — find it in the venues collection
organizer_uid	string	Your organizer user UID — find it in Firebase Console → Authentication → Users
total_registrations	number	0
total_revenue	number	0
total_checkins	number	0

- Click Save — your test event is now in Firestore
- Now add a ticket type — click the event document → click + Start collection
- Collection ID: ticket_types → Auto-ID → add these fields:

Field	Type	Value
name	string	Early Bird
price	number	50
quantity_total	number	100
quantity_sold	number	0
max_per_order	number	4
is_active	boolean	true

Step 4 — Rebuild Docker and test

```
docker compose down
docker compose build
docker compose up
```

Test every flow in the browser:

URL / Action	What to test	Expected result
http://localhost:5000/	Home page	Event card grid shows TechConf 2026 with date and venue

Search for 'TechConf'	Search filter	Card stays visible, other events hidden
Search for 'xyz'	Empty search	No events found message appears
Click Clear on search	Clear filter	All events shown again
Click View Details	Event detail page	Full event info, description, ticket type card
Ticket card shows Early Bird	Ticket display	Price ₹50, 100 tickets left, Max 4 per person
Logged out — card button	Login gate	Button shows Login to Register
Log in and return to home	Logged in state	Button changes to Register
Click Select & Register	Registration link	Currently 404 — registration route comes Day 9, that is expected
Promo code input	UI check	Typing a code and clicking Apply shows info message

The 404 on Select & Register is expected

The registration route (registration.register) does not exist yet — it is built on Day 9. The 404 today simply confirms the link is wired correctly. It will work on Day 9.

Day 5 deliverable: Public home page shows real event cards from Firestore. Event detail page shows ticket types with availability. Search works. Login gate working.

Step 5 — Commit

```
git add .
git status   # confirm secrets/ and .env not listed
git commit -m "Day 5: public event listing, event detail page, ticket type
cards"
git push origin develop
```

End-of-Day 5 Checklist

	Item	How to verify
☐	**app/routes/public.py replaced**	File has index() and event_detail() routes reading from Firestore
☐	**templates/public/index.html replaced**	Shows event cards grid with search bar and empty state
☐	**templates/public/event_detail.html created**	Shows event info, ticket type cards, promo code input
☐	**ems.css updated**	Added .ems-event-img, .ems-event-img-placeholder, .ems-ticket-card styles
☐	**Firestore index created**	events collection index (status + start_datetime) shows Enabled
☐	**Test event created in Firestore**	TechConf 2026 document exists in events collection with status=published
☐	**Ticket type created**	ticket_types subcollection has Early Bird document with is_active=true
☐	**Docker builds cleanly**	docker compose build completes without errors
☐	**Home page shows event card**	TechConf 2026 card visible at localhost:5000 with date and venue
☐	**Search works**	Searching by name filters cards correctly
☐	**Event detail page loads**	Clicking View Details shows full event info and ticket type card
☐	**Ticket availability shown**	Early Bird shows 100 tickets left, Max 4 per person
☐	**Login gate works**	Logged out users see Login to Register button
☐	**Logged in state**	Logged in users see Select & Register button
☐	**Committed & pushed**	GitHub develop branch shows Day 5 commit

What's Next — Day 6 Preview

Tomorrow you build the event creation and management pages for organizers — wireframe page 7. This is the day the organizer dashboard becomes real.

- Event create/edit form with venue dropdown (populated from Firestore venues)
- Save as draft and Publish buttons
- Organizer dashboard shows their events with registration counts
- Event status management (draft / published / cancelled)

- Update app/utils/validators.py with validate_event() logic

Tag and backup reminder
After completing Day 5:

```
git tag v0.0.5-day5 -m 'Day 5 complete — public event listing and detail pages'
git push origin v0.0.5-day5
cp -r /d/projects/event-management-system /d/projects/event-management-system-day5-backup
```

Day 6 — Setup Guide

Event Creation & Organizer Dashboard

Wireframes 6 & 7 | 5 Parts | ~6 hours

Part A	Update validators.py — add validate_event() with full checks (~30 min)
Part B	Write app/routes/events.py — create, edit, publish, delete routes (~90 min)
Part C	Write templates/organizer/events/ — form and list templates (~60 min)
Part D	Update organizer dashboard — replace placeholder with real data (~45 min)
Part E	Wire up, create Firestore indexes, rebuild Docker, test all flows (~30 min)
End goal	Organizer can create, publish, edit and delete events. Dashboard shows live metrics.

Overview — What we are building today

Today you build the organizer's event management features — wireframe pages 6 and 7. After today, the organizer no longer needs to create events manually in Firebase Console. They can create, edit, publish, and delete events entirely from the web app.

Wireframe	URL	What it does
Page 6 — Organizer Dashboard	/organizer/dashboard	Lists organizer's events with registrations, revenue, check-in counts
Page 7 — Create Event	/organizer/events/create	Form: name, description, dates, venue dropdown, status
Page 7 — Edit Event	/organizer/events/<id>/edit	Same form pre-filled with existing data
— Delete Event	POST /organizer/events/<id>/delete	Soft-delete: sets status to cancelled
— Publish Event	POST /organizer/events/<id>/publish	Changes status from draft to published

New files today

```
event-management-system/
├── app/
│   ├── routes/
│   │   └── events.py              ← NEW — event CRUD routes
│   └── utils/validators.py         ← UPDATE — add validate_event()
├── templates/organizer/
│   ├── dashboard.html             ← REPLACE placeholder with real data
│   └── events/                    ← NEW folder
│       ├── form.html              ← NEW — shared create/edit form
│       └── list.html              ← NEW — event list for organizer
└── app.py                         ← UPDATE — register events blueprint
```

Part A

Update app/utils/validators.py — Add validate_event()

Open app/utils/validators.py in VS Code. Replace the validate_event stub function with this full implementation:

```
from datetime import datetime

def validate_event(data):
    """
    Validate event form data.
    Returns a list of error strings. Empty = passed.
    """
    errors = []

    name        = data.get('name','').strip()
    description = data.get('description','').strip()
    start_str   = data.get('start_datetime','').strip()
    end_str     = data.get('end_datetime','').strip()
    venue_id    = data.get('venue_id','').strip()
    event_type  = data.get('event_type','').strip()

    # Name
    if not name:
        errors.append('Event name is required.')
    elif len(name) < 3:
        errors.append('Event name must be at least 3 characters.')
    elif len(name) > 150:
        errors.append('Event name must be under 150 characters.')

    # Description
    if not description:
        errors.append('Description is required.')

    # Venue
    if not venue_id:
        errors.append('Please select a venue.')

    # Event type
    if event_type not in ['physical', 'virtual', 'hybrid']:
        errors.append('Please select a valid event type.')

    # Dates — parse and compare
    start_dt = end_dt = None
    fmt = '%Y-%m-%dT%H:%M'  # HTML datetime-local format
    if not start_str:
        errors.append('Start date and time is required.')
    else:
        try:
            start_dt = datetime.strptime(start_str, fmt)
        except ValueError:
            errors.append('Invalid start date format.')

    if not end_str:
        errors.append('End date and time is required.')
```

```
    else:
        try:
            end_dt = datetime.strptime(end_str, fmt)
        except ValueError:
            errors.append('Invalid end date format.')

    if start_dt and end_dt:
        if end_dt <= start_dt:
            errors.append('End date must be after start date.')

    return errors
```

Part B

Write app/routes/events.py — Event CRUD Routes

Create a new file app/routes/events.py in VS Code. This handles all organizer event management — create, edit, publish, delete.

```
from flask import (Blueprint, render_template, redirect,
                        url_for, session, flash, request)
from app.firebase_config import db
from app.decorators import login_required, role_required
from app.utils.validators import validate_event
from datetime import datetime
from google.cloud.firestore import SERVER_TIMESTAMP

events_bp = Blueprint('events', __name__, url_prefix='/organizer/events')

FMT = '%Y-%m-%dT%H:%M'  # datetime-local HTML input format


# ── Helper: get event + verify ownership ──────────────────────────
def get_event_or_403(event_id):
    doc = db.collection('events').document(event_id).get()
    if not doc.exists:
        return None, None
    data = doc.to_dict()
    if data.get('organizer_uid') != session.get('uid'):
        return None, 'forbidden'
    return doc, data


# ── Helper: get organizer's venues for dropdown ──────────────────────
def get_my_venues():
    docs = (
        db.collection('venues')
        .where('organizer_uid','==',session['uid'])
        .order_by('name')
        .stream()
    )
    return [{**d.to_dict(),'id':d.id} for d in docs]


# ── LIST organizer's events ──────────────────────────────────────────
@events_bp.route('/')
@login_required
@role_required('organizer')
def list_events():
    docs = (
        db.collection('events')
        .where('organizer_uid','==',session['uid'])
        .order_by('created_at')
        .stream()
    )
    events = [{**d.to_dict(),'id':d.id} for d in docs]
    return render_template('organizer/events/list.html',events=events)
```

```
# ── CREATE event ──────────────────────────────────────────────────────
@events_bp.route('/create',methods=['GET','POST'])
@login_required
@role_required('organizer')
def create_event():
    venues = get_my_venues()

    if request.method == 'POST':
        form_data = request.form.to_dict()
        action    = form_data.pop('action','draft')  # 'draft' or 'publish'
        errors    = validate_event(form_data)

        if errors:
            for e in errors: flash(e,'danger')
            return render_template('organizer/events/form.html',

form_data=form_data,venues=venues,action='create')

        status = 'published' if action == 'publish' else 'draft'
        start_dt = datetime.strptime(form_data['start_datetime'],FMT)
        end_dt   = datetime.strptime(form_data['end_datetime'],FMT)

        _,ref = db.collection('events').add({
            'name':           form_data['name'].strip(),
            'description':    form_data['description'].strip(),
            'start_datetime': start_dt,
            'end_datetime':   end_dt,
            'venue_id':       form_data['venue_id'],
            'event_type':     form_data.get('event_type','physical'),
            'status':         status,
            'organizer_uid':  session['uid'],
            'total_registrations': 0,
            'total_revenue':       0,
            'total_checkins':      0,
            'created_at':     SERVER_TIMESTAMP,
            'published_at':   SERVER_TIMESTAMP if status=='published' else
None,
        })

        flash(f"Event '{form_data['name']}' {status}!",'success')
        return redirect(url_for('events.list_events'))

    return render_template('organizer/events/form.html',
                           form_data={},venues=venues,action='create')


# ── EDIT event ────────────────────────────────────────────────────────
@events_bp.route('/<event_id>/edit',methods=['GET','POST'])
@login_required
@role_required('organizer')
def edit_event(event_id):
    doc,data = get_event_or_403(event_id)
    if doc is None:
        flash('Event not found or access denied.','danger')
        return redirect(url_for('events.list_events'))

    venues = get_my_venues()

    # Convert timestamps to string for form pre-fill
```

```
    if data.get('start_datetime'):
        data['start_datetime'] = data['start_datetime'].strftime(FMT)
    if data.get('end_datetime'):
        data['end_datetime'] = data['end_datetime'].strftime(FMT)

    if request.method == 'POST':
        form_data = request.form.to_dict()
        form_data.pop('action',None)
        errors = validate_event(form_data)

        if errors:
            for e in errors: flash(e,'danger')
            return render_template('organizer/events/form.html',
                                   form_data=form_data,venues=venues,
                                   action='edit',event_id=event_id)

        start_dt = datetime.strptime(form_data['start_datetime'],FMT)
        end_dt   = datetime.strptime(form_data['end_datetime'],FMT)

        db.collection('events').document(event_id).update({
            'name':           form_data['name'].strip(),
            'description':    form_data['description'].strip(),
            'start_datetime': start_dt,
            'end_datetime':   end_dt,
            'venue_id':       form_data['venue_id'],
            'event_type':     form_data.get('event_type','physical'),
            'updated_at':     SERVER_TIMESTAMP,
        })
        flash(f"Event '{form_data['name']}' updated!",'success')
        return redirect(url_for('events.list_events'))

    return render_template('organizer/events/form.html',
                           form_data=data,venues=venues,
                           action='edit',event_id=event_id)


# ── PUBLISH event ──────────────────────────────────────────────────────
@events_bp.route('/<event_id>/publish',methods=['POST'])
@login_required
@role_required('organizer')
def publish_event(event_id):
    doc,data = get_event_or_403(event_id)
    if doc is None:
        flash('Event not found.','danger')
        return redirect(url_for('events.list_events'))

    db.collection('events').document(event_id).update({
        'status':       'published',
        'published_at': SERVER_TIMESTAMP
    })
    flash(f"Event '{data['name']}' is now published!",'success')
    return redirect(url_for('events.list_events'))


# ── DELETE event (soft delete — sets status to cancelled) ──────────────
@events_bp.route('/<event_id>/delete',methods=['POST'])
@login_required
@role_required('organizer')
def delete_event(event_id):
    doc,data = get_event_or_403(event_id)
```

```
    if doc is None:
        flash('Event not found.','danger')
        return redirect(url_for('events.list_events'))

    # Soft delete: mark as cancelled (preserves registration records)
    db.collection('events').document(event_id).update({
        'status': 'cancelled'
    })
    flash(f"Event '{data['name']}' has been cancelled.",'info')
    return redirect(url_for('events.list_events'))
```

Why soft delete instead of hard delete

If an event has registrations, hard-deleting it would leave orphaned registration records with no event to reference. Soft delete (setting status to 'cancelled') keeps the data intact for reporting and attendee records. Permanent deletion can be done by an admin from the Firebase Console if truly needed.

Part C

Write the 2 Event Organizer Templates

First create the folders and files in Git Bash:

```
mkdir -p templates/organizer/events
touch templates/organizer/events/form.html
templates/organizer/events/list.html
```

File 1 — templates/organizer/events/form.html

Open the file and paste:

```
{% extends 'base.html' %}
{% block title %}{{ 'Edit' if action=='edit' else 'Create' }} Event{%
endblock %}

{% block content %}
<div class="container py-4">
  <div class="row justify-content-center">
    <div class="col-lg-8">

      {# Breadcrumb #}
      <nav aria-label="breadcrumb" class="mb-3"><ol class="breadcrumb">
        <li class="breadcrumb-item"><a href="{{ url_for('events.list_events')
}}">My Events</a></li>
        <li class="breadcrumb-item active">{{ 'Edit Event' if action=='edit'
else 'Create New Event' }}</li>
      </ol></nav>

      <div class="ems-card card p-4">
        <div class="d-flex justify-content-between align-items-center mb-4">
          <h4 class="fw-bold mb-0">
            <i class="bi bi-calendar-plus me-2 text-primary"></i>
            {{ 'Edit Event' if action=='edit' else 'Create New Event' }}
          </h4>
        </div>

        {# Wireframe page 7: Save Draft + Publish buttons at top right #}
        {% if action == 'edit' %}
          <form method="POST" action="{{ url_for('events.edit_event',
event_id=event_id) }}" novalidate>
        {% else %}
          <form method="POST" action="{{ url_for('events.create_event') }}"
novalidate>
        {% endif %}
          <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">

          {# Event Name #}
          <div class="mb-3">
            <label class="form-label fw-semibold">Event Name <span
class="text-danger">*</span></label>
            <input type="text" name="name" class="form-control"
```

```
                    placeholder="e.g. TechConf 2026"
                    value="{{ form_data.get('name','') }}" required>
          </div>

          {# Description #}
          <div class="mb-3">
            <label class="form-label fw-semibold">Description <span
class="text-danger">*</span></label>
            <textarea name="description" class="form-control" rows="4"
                      placeholder="Describe your event...">{{
form_data.get('description','') }}</textarea>
          </div>

          {# Start and End dates #}
          <div class="row">
            <div class="col-md-6 mb-3">
              <label class="form-label fw-semibold">Start Date & Time <span
class="text-danger">*</span></label>
              <input type="datetime-local" name="start_datetime" class="form-
control"
                     value="{{ form_data.get('start_datetime','') }}"
required>
            </div>
            <div class="col-md-6 mb-3">
              <label class="form-label fw-semibold">End Date & Time <span
class="text-danger">*</span></label>
              <input type="datetime-local" name="end_datetime" class="form-
control"
                     value="{{ form_data.get('end_datetime','') }}" required>
            </div>
          </div>

          {# Venue dropdown + Event type #}
          <div class="row">
            <div class="col-md-7 mb-3">
              <label class="form-label fw-semibold">Venue <span class="text-
danger">*</span></label>
              <select name="venue_id" class="form-select" required>
                <option value="">— Select a venue —</option>
                {% for v in venues %}
                <option value="{{ v.id }}"
                  {{ 'selected' if form_data.get('venue_id')==v.id }}>
                  {{ v.name }}, {{ v.city }}
                </option>
                {% endfor %}
              </select>
              <div class="form-text">
                <a href="{{ url_for('venues.create_venue') }}"
target=" blank">+ Add new venue</a>
              </div>
            </div>
            <div class="col-md-5 mb-3">
              <label class="form-label fw-semibold">Event Type <span
class="text-danger">*</span></label>
              <select name="event_type" class="form-select">
                {% for t in ['physical','virtual','hybrid'] %}
                <option value="{{ t }}"
                  {{ 'selected' if form_data.get('event_type','')==t }}>
                  {{ t | capitalize }}
                </option>
                {% endfor %}
```

```
              </select>
            </div>
          </div>

          {# Action buttons — wireframe page 7 shows Save Draft + Publish #}
          <hr class="my-4">
          <div class="d-flex gap-2 flex-wrap">
            {% if action == 'create' %}
            <button type="submit" name="action" value="draft" class="btn btn-
outline-secondary">
              <i class="bi bi-pencil me-1"></i>Save as Draft
            </button>
            <button type="submit" name="action" value="publish" class="btn
btn-primary">
              <i class="bi bi-globe me-1"></i>Save & Publish
            </button>
            {% else %}
            <button type="submit" class="btn btn-primary">
              <i class="bi bi-check-lg me-1"></i>Save Changes
            </button>
            {% endif %}
            <a href="{{ url_for('events.list_events') }}" class="btn btn-
outline-secondary">Cancel</a>
          </div>

        </form>
      </div>
    </div>
  </div>
</div>
{% endblock %}
```

File 2 — templates/organizer/events/list.html

Open the file and paste:

```
{% extends 'base.html' %}
{% block title %}My Events{% endblock %}

{% block content %}
<div class="container py-4">
  <div class="d-flex justify-content-between align-items-center mb-4">
    <h2 class="fw-bold mb-0">My Events</h2>
    <a href="{{ url_for('events.create_event') }}" class="btn btn-primary">
      <i class="bi bi-plus-lg me-2"></i>Create New Event
    </a>
  </div>

  {% if not events %}
  <div class="ems-card card text-center py-5">
    <i class="bi bi-calendar-x fs-1 text-muted mb-3"></i>
    <h5 class="text-muted">No events yet</h5>
    <a href="{{ url_for('events.create_event') }}" class="btn btn-primary btn-sm mt-
2">Create your first event</a>
  </div>

  {% else %}
```

```
  {% for event in events %}
  <div class="ems-card card mb-3 p-3">
    <div class="d-flex justify-content-between align-items-start flex-wrap gap-2">
      <div>
        <h5 class="fw-bold mb-1">{{ event.name }}</h5>
        <p class="text-muted small mb-1">
          <i class="bi bi-calendar3 me-1"></i>
          {% if event.start_datetime %}{{ event.start_datetime.strftime('%b %d, %Y')
}}{% endif %}
        </p>
        {# Status badge #}
        {% set badge =
{'published':'success','draft':'secondary','cancelled':'danger','completed':'primary
'} %}
        <span class="badge bg-{{ badge.get(event.status,'secondary') }}">
          {{ event.status | capitalize }}
        </span>
      </div>

      {# Metrics — wireframe page 6 #}
      <div class="d-flex gap-3 text-center">
        <div><div class="fw-bold fs-5">{{ event.total_registrations or 0 }}</div>
             <div class="text-muted small">Registrations</div></div>
        <div><div class="fw-bold fs-5">₹{{ event.total_revenue or 0 }}</div>
             <div class="text-muted small">Revenue</div></div>
        <div><div class="fw-bold fs-5">{{ event.total_checkins or 0 }}</div>
             <div class="text-muted small">Check-ins</div></div>
      </div>

      {# Action buttons #}
      <div class="d-flex gap-2 flex-wrap">
        <a href="{{ url_for('public.event_detail', event_id=event.id) }}"
           class="btn btn-sm btn-outline-secondary" target="_blank">
          <i class="bi bi-eye me-1"></i>View
        </a>
        <a href="{{ url_for('events.edit_event', event_id=event.id) }}"
           class="btn btn-sm btn-outline-primary">
          <i class="bi bi-pencil me-1"></i>Edit
        </a>
        {% if event.status == 'draft' %}
        <form method="POST" action="{{ url_for('events.publish_event',
event_id=event.id) }}" class="d-inline">
          <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
          <button type="submit" class="btn btn-sm btn-success">
            <i class="bi bi-globe me-1"></i>Publish
          </button>
        </form>
        {% endif %}
        {% if event.status != 'cancelled' %}
        <form method="POST" action="{{ url_for('events.delete_event',
event_id=event.id) }}" class="d-inline"
              onsubmit="return confirm('Cancel event: {{ event.name }}?')">
          <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
          <button type="submit" class="btn btn-sm btn-outline-danger">
            <i class="bi bi-x-circle me-1"></i>Cancel
          </button>
        </form>
        {% endif %}
      </div>
    </div>
  </div>
```

```
    {% endfor %}
    {% endif %}
  </div>
  {% endblock %}
```

Part D

Update Organizer Dashboard — Replace Placeholder with Real Data

First update app/routes/organizer.py to pass real data to the dashboard. Open it and replace the entire file:

```
from flask import Blueprint, render_template, session
from app.firebase_config import db
from app.decorators import login_required, role_required

organizer_bp = Blueprint('organizer',__name__,url_prefix='/organizer')


@organizer_bp.route('/dashboard')
@login_required
@role_required('organizer')
def dashboard():
    # Fetch organizer's events
    docs = (
        db.collection('events')
        .where('organizer_uid','==',session['uid'])
        .order_by('created_at')
        .stream()
    )
    events = [{**d.to_dict(),'id':d.id} for d in docs]

    # Aggregate summary metrics across all events
    total_reg = sum(e.get('total_registrations',0) for e in events)
    total_rev = sum(e.get('total_revenue',0) for e in events)
    total_chk = sum(e.get('total_checkins',0) for e in events)
    published = sum(1 for e in events if e.get('status')=='published')

    return render_template('organizer/dashboard.html',
                           events=events,
                           total_reg=total_reg,
                           total_rev=total_rev,
                           total_chk=total_chk,
                           published=published)
```

Now open templates/organizer/dashboard.html and replace the entire placeholder:

```
{% extends 'base.html' %}
{% block title %}Dashboard{% endblock %}

{% block content %}
<div class="container py-4">

  {# Welcome header — wireframe page 6 #}
  <div class="mb-4">
```

```
    <h2 class="fw-bold">Welcome back, {{
session.get('email','Organizer').split('@')[0] | capitalize }}!</h2>
    <p class="text-muted">Here's an overview of all your events.</p>
  </div>

  {# Summary metric cards — wireframe page 6 #}
  <div class="row g-3 mb-4">
    {% for label,value,icon,color in [
      ('Total Events',   events|length,         'calendar-event', 'primary'),
      ('Published',      published,              'globe',          'success'),
      ('Registrations',  total_reg,              'people',         'info'),
      ('Revenue',        '₹'~total_rev,          'cash-stack',     'warning'),
      ('Check-ins',      total_chk,              'qr-code-scan',   'secondary'),
    ] %}
    <div class="col-6 col-md-4 col-lg-2">
      <div class="ems-card card text-center p-3 h-100">
        <i class="bi bi-{{ icon }} fs-3 text-{{ color }} mb-2"></i>
        <div class="fs-4 fw-bold">{{ value }}</div>
        <div class="text-muted small">{{ label }}</div>
      </div>
    </div>
    {% endfor %}
  </div>

  {# Events section #}
  <div class="d-flex justify-content-between align-items-center mb-3">
    <h4 class="fw-bold mb-0">Your Events</h4>
    <a href="{{ url_for('events.create_event') }}" class="btn btn-primary btn-sm">
      <i class="bi bi-plus-lg me-1"></i>Create New Event
    </a>
  </div>

  {% if not events %}
  <div class="ems-card card text-center py-5">
    <i class="bi bi-calendar-x fs-1 text-muted mb-3"></i>
    <h5 class="text-muted">No events yet</h5>
    <a href="{{ url_for('events.create_event') }}" class="btn btn-primary btn-sm mt-
2">Create your first event</a>
  </div>

  {% else %}
  {% for event in events %}
  <div class="ems-card card mb-3 p-3">
    <div class="d-flex justify-content-between align-items-center flex-wrap gap-2">
      <div>
        <h6 class="fw-bold mb-1">{{ event.name }}</h6>
        {% set
badge={'published':'success','draft':'secondary','cancelled':'danger','completed':'p
rimary'} %}
        <span class="badge bg-{{ badge.get(event.status,'secondary') }}">{{
event.status|capitalize }}</span>
      </div>
      <div class="d-flex gap-3 text-center">
        <div><div class="fw-bold">{{ event.total_registrations or 0 }}</div><div
class="text-muted small">Reg.</div></div>
        <div><div class="fw-bold">₹{{ event.total_revenue or 0 }}</div><div
class="text-muted small">Revenue</div></div>
        <div><div class="fw-bold">{{ event.total_checkins or 0 }}</div><div
class="text-muted small">Check-ins</div></div>
      </div>
      <div class="d-flex gap-2">
```

```
        <a href="{{ url_for('events.edit_event', event_id=event.id) }}" class="btn
btn-sm btn-outline-primary">
          <i class="bi bi-pencil me-1"></i>Manage</a>
        <a href="{{ url_for('public.event_detail', event_id=event.id) }}" class="btn
btn-sm btn-outline-secondary" target="_blank">
          <i class="bi bi-eye me-1"></i>View</a>
      </div>
    </div>
  </div>
  {% endfor %}
  {% endif %}
</div>
{% endblock %}
```

Part E

Wire Up, Firestore Indexes, Rebuild Docker & Test

Step 1 — Register events blueprint in app.py

Open app.py and add these two lines with the other blueprints:

```
from app.routes.events import events_bp
app.register_blueprint(events_bp)
```

Step 2 — Add Events link to organizer navbar in base.html

Open templates/base.html. Inside the organizer navbar block (after the Venues link), add:

```
<li class="nav-item">
  <a class="nav-link" href="{{ url_for('events.list_events') }}">
    <i class="bi bi-calendar-event me-1"></i>Events
  </a>
</li>
```

Step 3 — Create Firestore indexes

Two new queries today need composite indexes. Run docker compose up and visit the events list page. If index errors appear in the Docker logs, copy each URL and create the indexes:

Collection	Fields	Purpose
events	organizer_uid ASC + created_at ASC	Organizer event list and dashboard

Step 4 — Rebuild and test

```
docker compose down
docker compose build
docker compose up
```

Action	What to test	Expected result
Log in as organizer	Visit /organizer/dashboard	Dashboard shows metric cards (all 0s) and Create Event button
Click Create New Event	Form loads	Name, description, dates, venue dropdown, event type
Submit empty form	Validation	Flash: Event name is required

End date before start	Validation	Flash: End date must be after start date
Fill form, click Save as Draft	Creates draft	Flash: Event drafted! Appears in list with Draft badge
Fill form, click Save & Publish	Creates published	Flash: Event published! Appears in list with Published badge
Click Edit on an event	Edit form	Form pre-filled with existing data
Change name and save	Updates event	Flash: Event updated! New name shows in list
Click Publish on a draft	Publishes it	Draft badge changes to Published badge
Click Cancel on an event	Soft delete	Confirmation dialog, then Cancelled badge in list
Visit localhost:5000	Public home	Newly published event appears as a card
Dashboard metrics	Counts correct	Event count increments after creating events

Day 6 deliverable: Organizer can create, edit, publish, and cancel events entirely from the web app. Dashboard shows real event data. Events appear immediately on the public home page.

Step 5 — Commit

```
git add .
git status   # confirm secrets/ and .env not listed
git commit -m "Day 6: event CRUD, organizer dashboard with real data, publish
flow"
git push origin develop
```

End-of-Day 6 Checklist

	Item	How to verify
☐	**validate_event() implemented**	validators.py has full validation including date comparison
☐	**app/routes/events.py created**	Has list, create, edit, publish, delete routes all with route guards
☐	**templates/organizer/events/form.html**	Single form handles create and edit. Has Save Draft + Publish buttons
☐	**templates/organizer/events/list.html**	Shows events with status badges and metrics
☐	**organizer/dashboard.html replaced**	Shows metric cards and events list with real Firestore data
☐	**app/routes/organizer.py updated**	Passes events, total_reg, total_rev, total_chk, published to template
☐	**events_bp registered in app.py**	Blueprint imported and registered
☐	**Events link in organizer navbar**	Navbar shows Events link for organizer role
☐	**Firestore index created**	events (organizer_uid + created_at) index shows Enabled
☐	**Create event works**	New event saved to Firestore with all fields
☐	**Save as Draft works**	Event created with status=draft, not visible on public page
☐	**Save & Publish works**	Event created with status=published, visible on public page
☐	**Edit event works**	Changes persisted in Firestore
☐	**Publish draft works**	Status changes from draft to published
☐	**Cancel event works**	Status changes to cancelled, confirmation dialog shown
☐	**Dashboard metrics**	Event count and metric cards showing correct data
☐	**Committed & pushed**	GitHub develop branch shows Day 6 commit

What's Next — Day 7 Preview

Tomorrow you build the complete end of Week 1 — this is the last day of Sprint 1. You'll add the ticket type management pages for organizers (wireframe page 8) so they can add Early Bird, General Admission, and VIP tickets to their events.

- Ticket type create/edit/delete — subcollection under each event
- Organizer ticket management page — table with price, quantity, sold count
- Link from event edit page to ticket management
- validate_ticket_type() in validators.py
- Sprint 1 complete — end of Week 1

Tag and backup reminder

After completing Day 6:

```
git tag v0.0.6-day6 -m 'Day 6 complete — event CRUD, organizer dashboard live'
git push origin v0.0.6-day6
cp -r /d/projects/event-management-system /d/projects/event-management-system-day6-backup
```